Imperial College London

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Harnessing the Potential for Collective Intelligence with Large Language Models

Author: Pedro Urbina Rodriguez

Supervisor: Dr Francesco Belardinelli

Submitted in partial fulfillment of the requirements for the MSc degree in Advanced Computing of Imperial College London

September 2024

Abstract

Large Language Models (LLMs) represent a significant step toward achieving Artificial General Intelligence (AGI). Despite their impressive problem-solving capabilities in text-based tasks, LLMs still face several limitations that hinder their broader adoption. These challenges include their tendency to hallucinate, limited reasoning abilities, and constraints on context window length, among others.

To address these issues, the field of multi-agent LLM frameworks has emerged, integrating concepts from Multi-Agent Systems (MAS) with LLMs. In this work, we provide an overview of the key developments leading to the rise of multi-agent LLMs and offer a categorization of existing frameworks in the field.

The primary contribution of this thesis is the introduction of a meta-framework for multi-agent LLM systems, designed to enhance frameworks tasked with solving problems that have verifiable solutions. Through rigorous testing, we demonstrate that this meta-framework consistently improves the performance of two multi-agent LLM frameworks across different domains: mathematics and coding.

Lastly, we present a vision for the future of multi-agent LLMs, advocating for a shift toward the development of LLM organizations. Drawing parallels with the evolution of MAS, where the focus shifted from an agent-centered approach to an organization-centered approach by incorporating concepts from Organization Theory, we argue that similar advancements can lead to the creation of more capable and scalable LLM-based multi-agent systems.

Acknowledgments

First and foremost, I would like to express my deepest gratitude to my primary supervisor, Dr. Francesco Belardinelli, for agreeing to supervise my self-proposed project in the nascent research field of Multi-Agent Large Language Models. His guidance and support have been invaluable throughout the duration of this project.

I would also like to extend my sincere thanks to Professor Murray Shanahan. Our discussions over a year ago planted the seeds for ideas that ultimately became the foundation of this research. I am grateful for his insightful feedback during the preliminary stages and for his collaboration in the evaluation of this thesis.

I am deeply appreciative of OpenAI for providing computing resources through their Researcher Access Program. This project would not have been possible without their generous support, which enabled access to their most capable LLM models.

Finally, I would like to express my heartfelt gratitude to my family and friends. Their unwavering support and love throughout the development of this project have been absolutely key to its successful completion.

Contents

Chapter 1

Introduction

One of the holy grails in the field of Artificial Intelligence (AI) is the creation of Artificial General Intelligence (AGI)—an entity with problem-solving capabilities or intelligence at a level comparable to, or even surpassing, that of most human beings.

While it remains a subject of debate how close current AI systems are to achieving AGI, the emergence of Large Language Models (LLMs) is widely regarded as a significant step in that direction. LLMs have demonstrated unprecedented levels of generality in their problem-solving abilities across a wide range of text-based tasks, leading some to attribute them with *sparks of AGI* [1].

1.1 Problem Statement

While LLMs represent a significant advance in the generality of AI systems, they still face a range of substantial challenges. These include, but are not limited to:

- **Hallucinations:** Although LLMs learn and encapsulate vast amounts of factual knowledge during their training, their training objective is next-token prediction rather than ensuring factual accuracy. As a result, LLMs are prone to generating incorrect or misleading information, especially when operating outside their training distribution. This makes them unreliable for tasks that demand high levels of accuracy and factuality, limiting their broader adoption.
- **Reasoning Abilities:** While LLMs have demonstrated impressive performance across a wide variety of tasks, reasoning and logic remain areas with significant room for improvement. LLMs often struggle with tasks that require deep logical thinking and coherent reasoning.
- **Adaptive Compute:** Due to the autoregressive nature of token generation, LLMs typically use a similar amount of compute regardless of task complexity. Ideally, we would want LLMs to allocate more computational resources for complex tasks, much like how humans spend more time and effort on difficult problems compared to simpler ones.
- **Context Window Length:** Although the size of context windows has been increasing, LLM performance tends to decline as prompts grow longer. Managing long-horizon tasks remains a challenge, and simply increasing the context window size does not seem to be a sufficient solution.
- **Effective Cooperation between LLMs:** Multi-agent LLM systems have emerged as a

potential solution to overcome some of these limitations. However, a new challenge arises: How can we enable effective cooperation between multiple LLM agents? While several approaches have been proposed, the field of multi-agent LLMs is still in its infancy and requires further development.

1.2 Proposed Solution

To address the limitations of current LLMs, we adopt a multi-agent LLM approach. Specifically, this approach can help mitigate the previously identified issues in the following ways:

- **Hallucinations:** A multi-agent LLM system can implement error-correction, refinement, and voting mechanisms among different agents to minimize the impact of hallucinations. By cross-verifying information, agents can collaboratively refine outputs and reduce the risk of incorrect information.
- **Reasoning Abilities:** LLMs often struggle with complex reasoning tasks that require multiple reasoning steps. In a multi-agent LLM system, complex problems can be decomposed into smaller, simpler subproblems that individual agents can more effectively tackle. This iterative, step-by-step process increases the likelihood of successfully solving complex reasoning tasks.
- **Adaptive Compute:** A multi-agent approach allows for an implicit form of adaptive compute. By breaking down a problem and solving it iteratively through multiple agents, the system allocates more resources to harder problems, effectively simulating the way humans spend more time and effort on complex challenges.
- **Context Window Length:** By dividing a problem into smaller subproblems and assigning different agents to work on specific subtasks, the context required by each individual agent is reduced. This distributed representation across multiple agents allows the system to handle longer-horizon tasks without being constrained by the limitations of a single agent's context window.

To address the challenge of efficient collaboration in multi-agent LLMs, we focus on solving complex mathematics and coding problems. We build upon and enhance two recent multi-agent architectures, MACM [2] and AgentCoder [3], by introducing a meta-architecture. This meta-architecture is general enough to be applied to both mathematics and coding problems, demonstrating its flexibility and versatility. We further show how instantiating this architecture with MACM and AgentCoder improves their performance on the MATH [4] and HumanEval [5] benchmarks.

Additionally, we present our long-term vision for the field of multi-agent LLMs in Chapter 6. In this more conceptual chapter, we advocate for a shift from an agent-centered approach to an organization-centered approach in the design of multi-agent LLM systems. In this new paradigm, the focus moves away from the individual agent and towards the system as a whole, with the organizational structure and mechanisms at the forefront of design priorities. We draw inspiration from a similar shift that occurred two decades ago in the field of pure Multi-Agent Systems (MAS), transitioning from an agent-centered approach (ACMAS) to an organization-centered approach (OCMAS). This shift gave rise to the field of Multi-Agent Organizations, situated at the intersection of MAS and Organization Theory.

1.3 Contributions

Our main contributions can be summarized as follows:

- We provide an in-depth literature review of the field of Multi-Agent LLM frameworks, offering a critical overview of the current state and trajectory of this rapidly emerging and evolving field.
- We design and implement AgentCoder+, an extension of the original AgentCoder framework, and introduce a meta-framework applicable to any problem-solving task whose solution can be tested. We apply this meta-framework to the AgentCoder+ and MACM architectures, resulting in the creation of MetaMACM and MetaAgentCoder+ multi-agent LLM architectures.
- We empirically validate the effectiveness of AgentCoder+, MetaMACM, and MetaAgentCoder+ on the HumanEval and MATH datasets, demonstrating improvements in both coding and mathematical reasoning capabilities of LLMs.
- We propose a vision for the future of multi-agent LLMs, advocating for a shift towards the development of LLM Organizations. We draw parallels to the transition from Agent-Centered MAS to Organization-Centered MAS in the field of MAS, which gave rise to the field of Multi-Agent Organizations by synergistically integrating MAS with Organization Theory.

Chapter 2

Background

This chapter provides essential background, offering a concise overview of deep learning's role in the pursuit of generally intelligent systems, a journey that has led to the development of pretrained Large Language Models. We also provide a brief introduction to the classical field of Multi-Agent Systems.

2.1 Deep Learning

One of the ultimate goals of the field of Artificial Intelligence is to achieve general intelligence. This quest initially led to the development of logic based systems, which suffered from immense complexity of the world, and the difficulty to formalize all of our implicit knowledge.

Learning can be conceptualized as the process of extracting and internalizing relevant patterns, relationships, and knowledge from a given set of experiences or data within a specific environment and task context. The ultimate goal of this process is to optimize performance on the task by effectively utilizing the learned information when faced with similar situations in the future, while also achieving a level of generalization that allows for robust performance in novel, but related, scenarios [6].

The problem of learning can be thought of as the problem of finding a suitable program, given a set of experiences or data, that generates outputs to maximize performance on a particular task. For any given set of data or experiences, there exists an infinite number of programs that can fit these observations. However, the key challenge lies not only in fitting the data but also in achieving the ability to generalize to novel, unseen situations. It is both intuitively and formally recognized that shorter programs tend to generalize better [7, 8, 9, 10].

In the quest for an effective learning algorithm, one would ideally seek to explore the entire space of possible programs. However, this space is extraordinarily vast, making a comprehensive search infeasible in practice. Furthermore, the requirement to find the shortest program that generalizes well—critical for achieving general intelligence—adds an additional layer of complexity. The challenge lies in efficiently navigating this immense space to identify programs that not only fit the data but also exhibit strong generalization capabilities, all within computationally feasible bounds.

A common strategy in such circumstances is to constrain the search space to one that is sufficiently general yet more tractable. Deep learning emerges as a powerful approach lying

at the intersection of the space of sufficiently general programs and the space of optimizable programs. The generality of deep learning is supported by the Universal Approximation Theorem [11, 12], while its optimizability is enabled by the backpropagation algorithm [13].

Initially, deep learning faced challenges due to computational constraints. However, as Moore's Law continued to hold, enabling greater computational power, the immense potential of deep learning became apparent with breakthroughs such as AlexNet [14] in image classification and AlphaGo [15] in reinforcement learning. This has led to a point where, in the general public's perception, Artificial Intelligence, Machine Learning, and Deep Learning are often viewed as synonymous.

2.2 The Transformer Architecture

The interplay between hardware advancements and algorithm development is crucial to the evolution of deep learning. The rise of deep learning was largely driven by the availability of powerful GPUs capable of efficiently executing the backpropagation algorithm to train neural networks. Similarly, the advent of parallel computing, which allows for intense computational loads to be distributed across multiple processing units, necessitated the development of a new learning architecture to fully exploit this capability.

While neural networks achieved significant successes and remarkable levels of generality, their application remained somewhat constrained to specific domains, such as games and vision tasks. To approach human-level general intelligence, a key element was missing: scalability. The human brain consists of tens of billions of neurons, far surpassing the scale of systems like AlexNet and AlphaGo [14, 15]. Moreover, simply scaling existing neural network architectures was computationally infeasible.

In the pursuit of human-level general intelligence, scalability emerged as an essential requirement, in addition to the generality and optimizability that neural networks already provided. The Transformer architecture, introduced by Vaswani et al. in 2017 [16], met this new requirement by offering an inherently parallelizable structure. This, combined with the significant increase in computational power, paved the way for the development of Large Language Models.

2.2.1 Architectural Innovations

Transformers first emerged in the field of Natural Language Processing (NLP). The original *Attention is All You Need* paper [16], which introduced the transformer architecture, was initially focused on improving machine translation. However, the architecture's flexibility, scalability, and ability to capture long-range dependencies without the need for recurrence quickly transcended its original application in machine translation.

Before the advent of the Transformer architecture, deep learning models for machine translation and other NLP-related tasks primarily relied on Recurrent Neural Networks (RNNs) [13] and Long Short-Term Memory (LSTM) [17] networks, which are characterized by their sequential processing nature. The main drawback of these architectures was that their sequential data processing often led to difficulties in capturing long-range dependencies within the data.

The transformer architecture addressed this problem with the attention mechanism. Although the attention mechanism had been introduced previously [18, 19], Vaswani et al. [16] were the first to create an architecture that relied solely on attention, eliminating the

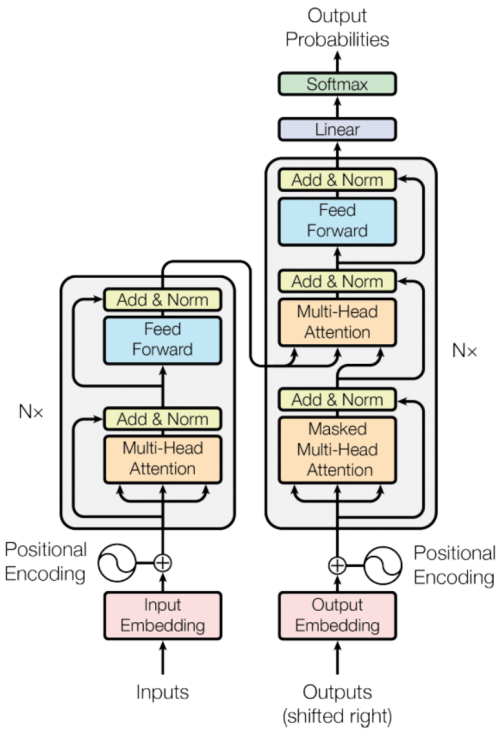


Figure 2.1: The original encoder-decoder Transformer architecture. Figure extracted from Vaswani et al. [16].

need for recurrence and convolutions. The attention mechanism works by dynamically computing a weighted sum of all elements in a sequence, allowing the model to focus on the most relevant parts. This ability to prioritize information is crucial for capturing long-range dependencies, making the transformer highly effective for tasks like translation and contextual understanding in language.

An high-level illustration of the transformer architecture is shown in Figure 2.1. The main architectural innovations of the transformer architecture include the following:

- Self-Attention Mechanism: While the original attention mechanism used two different sequences, the self-attention mechanism computes attention within a single sequence, enabling the transformer to represent the internal structure and dependencies of that sequence.

- Positional Encoding: Unlike RNNs and LSTMs, which inherently encode the order of sequences due to their sequential nature, the self-attention mechanism does not inherently encode sequence order. Positional encodings allow the transformer to receive information about the relative or absolute positions of tokens in a sequence.

- Multi-Head Attention: Instead of using a single attention mechanism (or attention head), transformers compute multiple attention distributions or projections simultaneously, which are then concatenated. This allows transformers to capture the multidimensional nature of dependencies and structures in the original sequence.

- Feed-Forward Neural Networks: The transformer includes feed-forward neural networks after each multi-head attention block at each layer. This introduces additional non-linearities, enabling more complex transformations of the original data and improving the architecture's overall expressiveness.

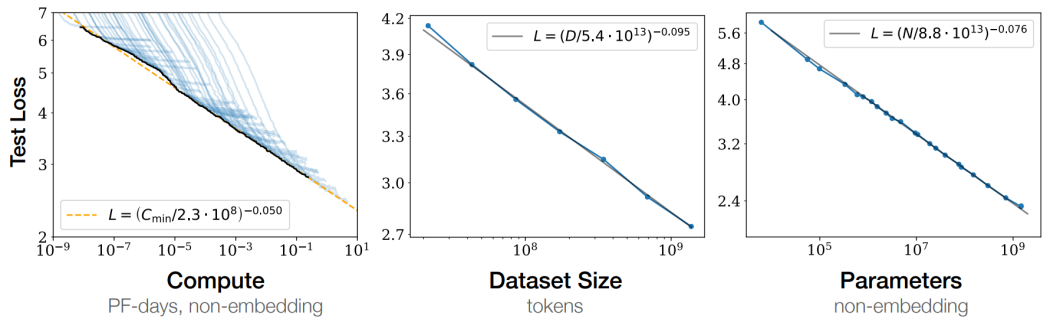


Figure 2.2: Scaling laws for language modeling. Performance smoothly improves when compute, dataset size, or the number of parameters are exponentially increased without being bottlenecked by the other two factors. Figure extracted from Kaplan et al. [21].

- Layer Normalization and Residual Connections: The architecture incorporates layer normalization and residual connections, allowing the architecture to be made deeper while remaining effectively trainable.

The combination of these innovations into a single architecture made the transformer extremely scalable and parallelizable to an unprecedented degree. Alongside hardware advancements defined by Moore's Law, this architecture has contributed to the widely recognized general intelligent capabilities of current large language models.

2.3 The Emergence of Large Language Models

Large Language Models (LLMs) represent a significant milestone in the progress of Artificial Intelligence towards achieving generally intelligent agents. Recent LLMs have even been noted for exhibiting *Sparks of Artificial General Intelligence* [1].

LLMs are primarily characterized by their use of the Transformer architecture and their immense scale, both in terms of parameters—ranging from billions to trillions—and in the volume of training data, often encompassing vast portions of the internet. This scale has endowed the models with an unprecedented level of fluidity and coherence in language generation, to the extent that they have rendered the Turing test [20] less relevant as a measure of general intelligence.

The advent of LLMs has led to a paradigm shift in Natural Language Processing (NLP), where many classical tasks such as translation, summarization, text generation, and even reasoning can now be performed by a single, general model, sometimes achieving superhuman performance. For these reasons, LLMs represent a significant step towards achieving general artificial intelligence.

2.3.1 The Role of Scaling Laws

The emergence and success of LLMs cannot be fully understood without considering the, at first glance, counterintuitive results of the *Scaling Laws* for language modeling, first identified by Kaplan et al. [21]. Prior to this, the prevailing wisdom in the field of AI suggested that very large neural models would primarily overfit the data, as the concept of overfitting was central to the discipline.

However, the scaling laws revealed that there is a non-naive approach to scaling models that not only avoids performance degradation but actually enhances it. This is achieved

by simultaneously increasing compute power, dataset size, and the number of parameters. When these factors are scaled together, power laws emerge in the language model's accuracy, with respect to compute, dataset size, and the number of parameters. This means that as we increase these three variables, we observe logarithmic improvements in prediction accuracy, which often correlates with enhanced performance across various benchmarks.

The scaling laws for language modeling are captured by the following equation:

$$\text{Loss} \propto \left(\frac{1}{N^{\alpha}} + \frac{1}{D^{\beta}} + \frac{1}{C^{\gamma}} \right)^{\delta}, \tag{2.1}$$

where N is the model size (number of parameters), D is the dataset size, C is the compute power, α, β, and γ are exponents that describe how sensitive the performance is to changes in these factors, respectively, and δ is a constant that adjusts the overall scale of the relationship.

Beyond the mathematics, the fundamental paradigm shift was the realization that, contrary to prior belief, there exists an optimal subspace in the space of possible model sizes, dataset sizes, and compute budgets. Within this subspace, scaling the model size in tandem with dataset size and compute power leads to predictably better results across a vast range of orders of magnitude. In other words, there is a direction in the space of possible model sizes, dataset sizes, and compute budgets along which performance improves predictably according to a power law. Practically, this meant that given a fixed amount of compute and dataset size, an optimal number of parameters can be determined to maximize the model's performance, leading to the rapid improvements observed in model capabilities.

2.3.2 Generalization and Emergent Capabilities

Alongside improvements in a language model's token prediction loss, emergent capabilities have appeared with each order of magnitude increase in model size, training data, and compute power. In this context, emergent capabilities refer to abilities acquired by the model that were not explicitly programmed into it. These capabilities are a byproduct of the model's scale, which directly enhances its ability to extract far more complex patterns from data than smaller models can.

These emergent capabilities can also be characterized as phase transitions in the model's abilities with respect to its test loss. For example, the transition from GPT-2 [22] to GPT-3 [23] represented a jump in model size from the order of a billion parameters to the order of hundreds of billions of parameters. This two-order-of-magnitude increase in model size resulted in emergent capabilities in GPT-3 that were only beginning to appear in GPT-2, or were completely absent. Some of these emergent capabilities include the ability to perform simple arithmetic, answer factual questions, create imaginative stories, and even write poetry.

Two of the most impressive emergent capabilities LLMs have demonstrated are the abilities to perform zero-shot and few-shot learning. Few-shot learning refers to the ability of models to perform tasks that they have never been explicitly trained on, given a few examples of how to solve the task. It turns out that models on the scale of GPT-3 and larger are able to perform quite well on new tasks using only the provided examples and their previously internalized data and generalization abilities. Zero-shot learning similarly requires a model to perform a task it has never been trained on, but this time without any examples of how to do it. For instance, Gemini 1.5 [24] is able to learn in context to translate the Kalamang language (a language it has never been trained on) to English, given only a Kalamang grammar manual.

If scaling laws continue to hold and emergent capabilities continue to appear as we further scale these Transformer-based models, human-level artificial general intelligence might

emerge sooner than expected. However, it remains to be seen whether new capabilities will emerge from the next-token prediction learning paradigm as we progress through further orders of magnitude.

2.4 Multi-Agent Systems

The aim of this work is to help bridge two seemingly distant research fields in Artificial Intelligence: Large Language Models and Multi-Agent Systems. Unlike the recently developed field of Large Language Models, the field of Multi-Agent Systems (MAS) has a rich history, with roots tracing back to the 1980s when it emerged as a concrete research area from the field of Distributed Artificial Intelligence. While these fields differ in history and development, this work aims to demonstrate the potentially synergistic relationship between these two subfields of Artificial Intelligence. To that end, we provide a brief overview of the field of Multi-Agent Systems.

2.4.1 Key Concepts and Components of Multi-Agent Systems

At its core, the field of MAS is an area of Artificial Intelligence that deals with systems composed of multiple autonomous entities, usually referred to as agents, that interact within a shared environment. Agents are designed to achieve either individual or collective goals through interaction with other agents, making this approach particularly useful for problems that require distributed problem-solving abilities. MAS addresses issues such as robustness, scalability, and coordination, which individual agents often cannot tackle on their own.

The foundations of MAS are deeply rooted in Game Theory, which provides a framework for studying the strategic interactions of agents. Concepts such as Nash equilibrium and Pareto optimality are central to the field of MAS. Beyond Game Theory, learning algorithms also play a vital role in MAS. In particular, reinforcement learning has been widely applied within MAS, giving rise to the specialized field of multi-agent reinforcement learning. Additionally, distributed problem-solving algorithms are key components in the effective functioning of MAS.

Agents are one of the core concepts of MAS. An agent is an autonomous entity capable of perceiving its environment, making decisions, and taking actions to achieve its objectives. Key characteristics of agents include autonomy and proactivity. Autonomy allows agents to take actions independently without external intervention, while proactivity refers to the initiative agents exhibit in pursuing their goals.

Another crucial element of MAS is the environment, which defines the rules and constraints that agents must adopt in order to interact and achieve their objectives. Additionally, interactions between agents are an essential component of MAS and can take the form of competition, cooperation, coordination, or negotiation, each playing a critical role in achieving the overall system's goals. The most common form of interaction is cooperation, where agents work together to achieve a goal that no single agent could accomplish on its own.

2.4.2 Types of Multi-Agent Systems

There are several dimensions along which Multi-Agent Systems (MAS) can be classified. The first dimension is homogeneous vs. heterogeneous MAS. In a homogeneous MAS, all agents are identical in terms of capabilities and objectives, resulting in simpler coordination strategies. On the other hand, a heterogeneous MAS consists of agents with distinct capabilities and goals, which necessitates more complex coordination protocols.

MAS can also be classified based on the level of agency centralization. In centralized MAS, a single agent oversees the overall functioning of the system, setting the goals and paths for the other agents to follow. In decentralized MAS, power and control are distributed among different agents, or may even emerge from decentralized cooperation. While this increases the complexity of the coordination problem, it also makes the system more robust to failures, avoiding the single point of failure that is often a drawback of the centralized approach.

Finally, MAS can be either static or dynamic. In dynamic MAS, agents are allowed to enter and leave the system, leading to interactions between agents that change over time and better represent the complexity of real-world environments. In contrast, static MAS have fixed agents, making their interactions more predictable.

2.4.3 Coordination and Collaboration Mechanisms

A key factor in the success of MAS is the design and implementation of effective coordination and collaboration mechanisms. Coordination refers to the system's ability to organize its agents so that they do not interfere with each other, enabling each agent to successfully achieve its goals. This can be accomplished through various strategies. Among the most common are task allocation, where each agent is assigned different and independent tasks, and scheduling, where each agent is given different timeframes to work on specific tasks.

Collaboration goes a step further than mere coordination, as it requires agents to work together toward the achievement of a common goal. This may involve the creation of teams or groups where agents synchronize their efforts by distributing tasks and sharing task-relevant information. In particular, communication is a critical component of MAS, essential for achieving effective coordination and collaboration through information exchange.

There are several dimensions along which communication protocols for MAS can be classified. However, one of the most important distinctions is whether communication is direct or indirect. Direct communication involves agents sending messages directly to each other. Indirect communication is usually achieved through stigmergy, where agents interact with one another via the environment, leaving signals that other agents can interpret.

Chapter 3

Literature Review

The intent of this chapter is to elucidate unprecedented potential that Large Language Models (LLMs) hold in the quest to achieve Collective Intelligence (CI) through artificial systems using a Multi-Agent Systems approach. To this end, we critically review a substantial body of recent work that begins to realize this potential. Our primary focus, however, is on identifying the main weaknesses and conceptual limitations currently present in this emerging field.

3.1 From LLMs to LLM Agents

The field of Large Language Models (LLMs) has experienced significant growth since the release of GPT-3 in 2020 [23]. This trend gained momentum with the release of ChatGPT in 2022 and further accelerated with the introduction of GPT-4, Gemini, Claude, and others in 2023 [25, 26]. These models have increasingly demonstrated emergent capabilities, such as in-context learning and common sense reasoning, and have exhibited generalization across a wide range of tasks due to their extensive and diverse training datasets. Their potential to eventually surpass human performance in various tasks, including planning and reasoning, underscores their transformative impact on the field of AI.

The concept of an *agent* has its origins in philosophy, with roots tracing back to thinkers like Aristotle and Hume. In the context of philosophy, an agent is an entity capable of action and possessing intentionality. This concept was later adopted by the fields of Computer Science and AI to denote computer programs that can understand users' goals and take autonomous actions to achieve them. Agents in this context are characterized by autonomy, proactiveness, and reactivity [27]. These characteristics make agents a key paradigm in the pursuit of Artificial General Intelligence (AGI).

The field of AI has shown particular interest in the development of agents. The central aim of AI research can be framed as the endeavor to build intelligent agents. In the latter half of the 20th century, the first AI agents were developed, focusing on narrow tasks such as symbolic reasoning and mastering specific games like Chess. Since 2015, the field of Reinforcement Learning (RL) has further popularized the term *agent*, achieving superhuman performance in games like Go [28] and hundreds of Atari games [28]. Advances in RL have led to the creation of more generally capable agents, such as MuZero [29] and Gato [30]. These increasingly generalist agents demonstrate pathways towards achieving human-level general intelligence.

LLMs have emerged as powerful candidates for serving as the backbone or brain for ad-

vanced AI agents. Their enormous training data and size have endowed these models with a generality far beyond what was imaginable a few years ago. They have demonstrated near-human abilities in many tasks and can effectively use a wide range of tools due to their exceptional natural language understanding, making them ideal candidates for achieving AGI.

The first attempts to create LLM-based agents include AutoGPT [31] and BabyAGI [32]. These LLM-based agents allow users to specify high-level tasks or goals, and the framework leverages the underlying LLM and prompt engineering techniques to autonomously solve the tasks. In particular, these frameworks start by breaking down the tasks into smaller, more manageable subtasks, which are then executed sequentially. Reflection mechanisms are also included, where the LLM reflects on the results and outcomes of the tasks, allowing it to iterate if problems are encountered or if the plan needs modification. One key characteristic of these frameworks is their inherently sequential execution of tasks, and they are often limited by their context window size.

3.2 From LLM Agents to Multi-Agent LLMs

While individual LLM agents demonstrate impressive cognitive capabilities due to the underlying power of LLMs, they still face several issues that significantly hinder their usefulness. LLMs lack a solid grounding in the real world, often referred to as a world model. This stems from their training objective of next-token prediction, where a world model can only emerge as an indirect consequence. This absence of a directly constructed, accurate world model leads to various problems, including the occasional production of incorrect information, commonly referred to as *hallucinations*. These issues, along with limitations in context length, impede the ability of individual LLM agents to tackle complex problems and engage in reasoning over extended time horizons. Additionally, LLMs are often biased towards complacency and self-consistency, which reduces the effectiveness of reflection mechanisms in individual agents.

To address these problems, the field of Multi-Agent LLMs has emerged. Drawing insights from Minsky's *Society of Mind* [33] and concepts like *cognitive synergy*, the focus has shifted to coordinating and orchestrating a swarm of LLM agents to enhance the problem-solving capabilities of the overall multi-agent system. Some of the benefits of transitioning from a single-agent paradigm to a multi-agent paradigm include:

- **Agent Specialization**: Even if the underlying LLM is the same for all agents, specialization of labor has been shown to be very effective in enhancing the problem-solving capabilities of human groups. Additionally, *profiling* agents differently through prompt engineering can improve the accuracy of models by triggering specialized regions of the LLM. Different agents can also be given access to various resources, tools, and other artifacts, and use different fine-tuned versions accordingly.

- **Context Length and Cognitive Space**: Using different agents to solve subtasks of a problem provides each agent with unique cognitive space to focus on specific parts of the problem. The limited context length of current models is a significant issue, but even when context length is not fully used, irrelevant previous context can negatively affect accuracy. Providing independent contexts for each agent is beneficial.

- **Robustness through Collective Decision Making**: LLMs are prone to hallucination and self-consistency, making self-supervision and correction challenging for a single agent. Using several independent agents enhances supervision, counteracting inherent

hallucination problems. Mechanisms such as majority voting can further improve robustness in decision-making processes, while competition mechanisms can select the best solutions to a given problem.

- **Hierarchical Task Decomposition**: Although it is theoretically possible to hierarchically decompose tasks to arbitrary depths using a single agent, it is more natural to use different agents for various types of tasks. This reduces context switching, which can potentially confuse agents, and avoids reaching context length limits where catastrophic forgetting may occur.
- **Parallelization**: Unlike single-agent LLMs, which operate as monolithic entities and often face limitations in scalability, Multi-Agent LLMs are inherently designed for parallel execution. By distributing tasks across multiple specialized agents, these systems can efficiently handle complex problems in a manner akin to collaborative human teams. This parallelization accelerates processing and enhances the system's ability to manage diverse and simultaneous tasks, thereby improving overall performance and scalability.

However, to fully harness these advantages of transitioning from individual to multi-agent settings, several new challenges emerge. These challenges can be summarized as the design of an *interaction layer* [34] on top of the *intelligence layer* provided by the LLMs. This *interaction layer* encompasses the design of collaboration, communication, cooperation, hierarchical decomposition, supervision, competition, and other mechanisms necessary to effectively coordinate these multi-agent LLM systems. In the next sections, we explore how current LLM frameworks have addressed these challenges and design decisions.

3.3 Multi-Agent Large Language Model Architectures

The aim of this section is to provide an overview of two of the currently most successful LLM multi-agent frameworks, AgentVerse [35] and MetaGPT [36], which will motivate the categorization of all current multi-agent LLM frameworks provided in the subsequent section.

3.3.1 AgentVerse

AgentVerse is a generalist multi-agent LLM framework created by Chen et al. [35]. Prior to AgentVerse, most agentic and multi-agent LLM frameworks were targeted toward specific tasks, which hindered their wide applicability. AgentVerse was among the first frameworks to aim at creating a generalist problem-solving approach. As such, AgentVerse is task-agnostic and can be applied to a wide range of problem-solving tasks.

The main components of the architecture are illustrated in Figure 3.1 and consist of four different stages. The first stage is the expert recruitment phase. Here, an LLM agent takes on the role of a human resource manager and is prompted to recruit several expert agents based on information about the task's nature. A key aspect of this phase is that it is performed automatically, without any human intervention in deciding which types of agents are required for the task, thus providing the system with significant flexibility and scalability.

Once the various expert agents are selected by the expert recruiter agent, the system moves into the second phase, collaborative decision-making. In this phase, the experts engage in a group discussion on the best way to proceed to achieve the task goal. This collaborative decision-making phase can be facilitated by either a horizontal democratic structure, where the outputs of every agent are equally integrated into the final decision, or a vertical structure, where the communication style includes a clear distinction of roles, such as feedback agents that induce iterative refinements.

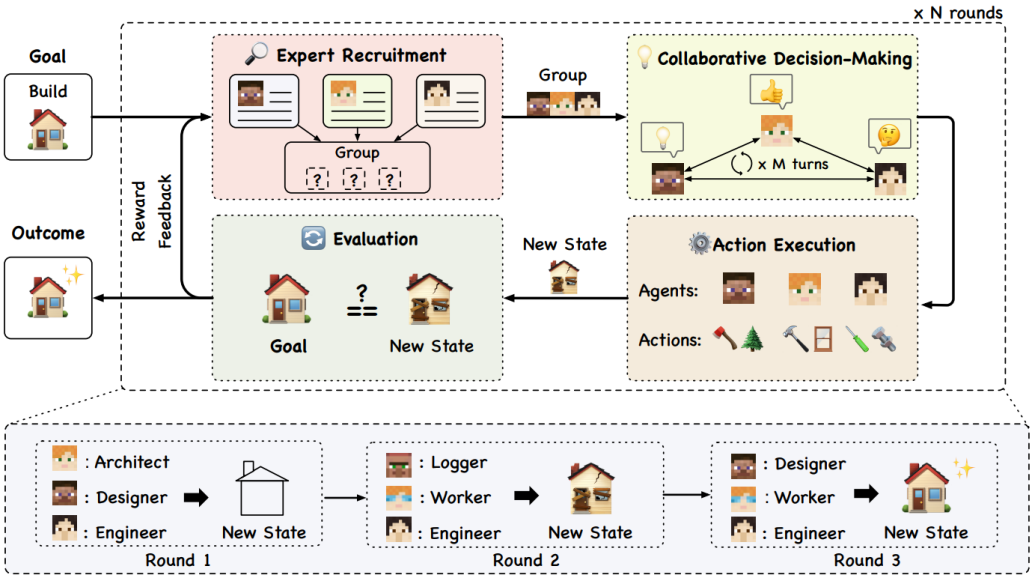


Figure 3.1: Illustration of the AgentVerse architecture. Figure extracted from [35].

The next phase is the action execution phase, where the different agents perform the agreed-upon actions, resulting in a change in the environment. This leads to the final evaluation phase, where the system assesses whether it has achieved its objective or goal. To assess this, a feedback mechanism must be designed, either by a human or by an additional LLM-based agent.

If the goal is not achieved, the system returns to the expert recruitment phase, but now with feedback on what is missing to reach the goal and how to act accordingly. This loop is repeated until the evaluation module outcome is satisfactory or a maximum number of iterations is reached.

The authors compare the performance of the AgentVerse framework against a simple Chain of Thought-prompted LLM [37], a *solo* version of the framework with a single agent, and the multi-agent version of the framework. They benchmark the performance across a variety of tasks, including creative writing, mathematical and logical puzzles, and coding. Their results show that the AgentVerse framework generally outperforms the simple Chain of Thought prompting strategy, with the multi-agent version consistently surpassing the *solo* version.

However, an interesting caveat is that the *solo* version of the framework sometimes outperforms the multi-agent framework when using less powerful models. The authors suggest that this might be due to erroneous feedback: even though the first agent may arrive at the correct solution initially, it can be confused by incorrect feedback, which ultimately hinders the overall performance of the system when using less capable models.

In addition, during embodied benchmarks using Minecraft, the authors report a variety of emergent phenomena. They observe volunteer behaviors, where some agents willingly contribute their time, resources, and even general assistance to other agents to achieve tasks. They also note conformity behaviors, where agents that have deviated from the group's mission are realigned with the group's plan, as well as destructive behaviors, where agents bypass agreed-upon safety procedures or even harm other agents in their own self-interest.

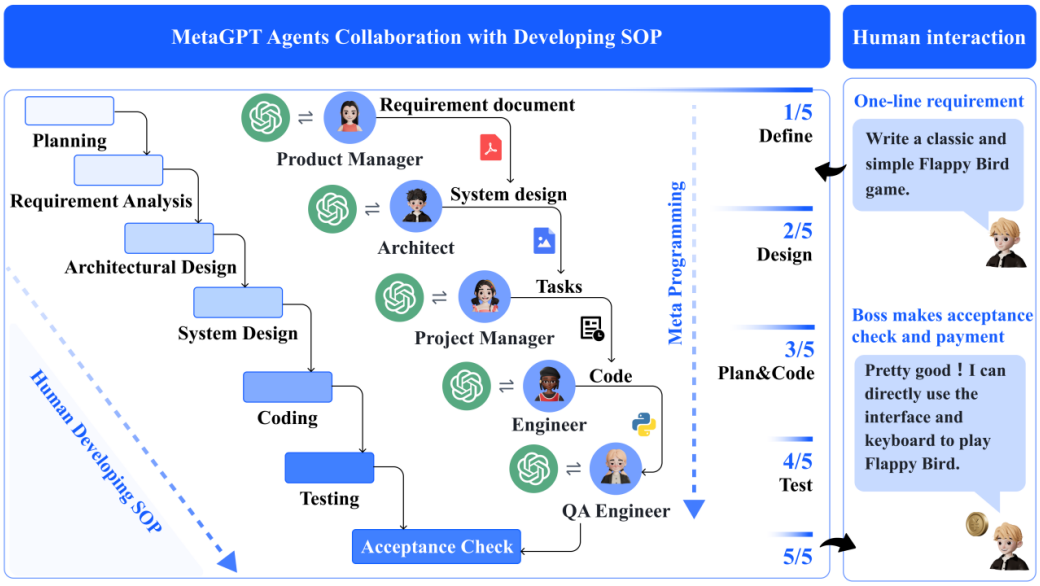


Figure 3.2: Illustration of the MetaGPT architecture. Figure extracted from [36].

3.3.2 MetaGPT

While AgentVerse aimed to create a generalist problem-solving framework, MetaGPT [36] takes the opposite approach by focusing on optimizing a multi-agent LLM framework specifically for coding and software engineering. The reasoning behind this focus is that, while a generalist problem-solving agent might be more desirable in the long run, the abstractness of a general framework can hinder the potential performance of a task-specific, fine-tuned multi-agent LLM architecture.

MetaGPT is an LLM-based meta-programming framework that harnesses the power of collaboration in software engineering. It leverages well-established standard operating procedures (SOPs) in software engineering to create a highly robust and reliable LLM-based multi-agent framework.

One of the key factors contributing to MetaGPT's success is its emphasis on role specialization. Complex problem-solving often requires breaking down large tasks into smaller, more manageable subtasks. In human endeavors, solving complex problems typically involves collaboration among actors with diverse backgrounds and expertise. By assigning each agent a specific context and skill set, MetaGPT mimics the collaborative processes observed in human organizations.

Figure 3.2 illustrates the five different agent types in MetaGPT. Specifically, the Product Manager is responsible for generating a requirement analysis document, which is then used by the Architect to transform user requirements into system design documents. Once the overall system is adequately designed, the Project Manager breaks down the system requirements into actionable tasks, which are then handled by a team of Engineers who perform the coding. Finally, the Quality Assurance Engineer conducts tests to verify that the code is functioning correctly.

Another key feature of the MetaGPT framework is its communication protocol, which is based on structured communication interfaces. While natural language can be an effective communication protocol, even humans design protocols to structure their communication to ensure no relevant information is overlooked and to avoid generating unnecessary information. MetaGPT imposes structured formats on agents depending on their respective

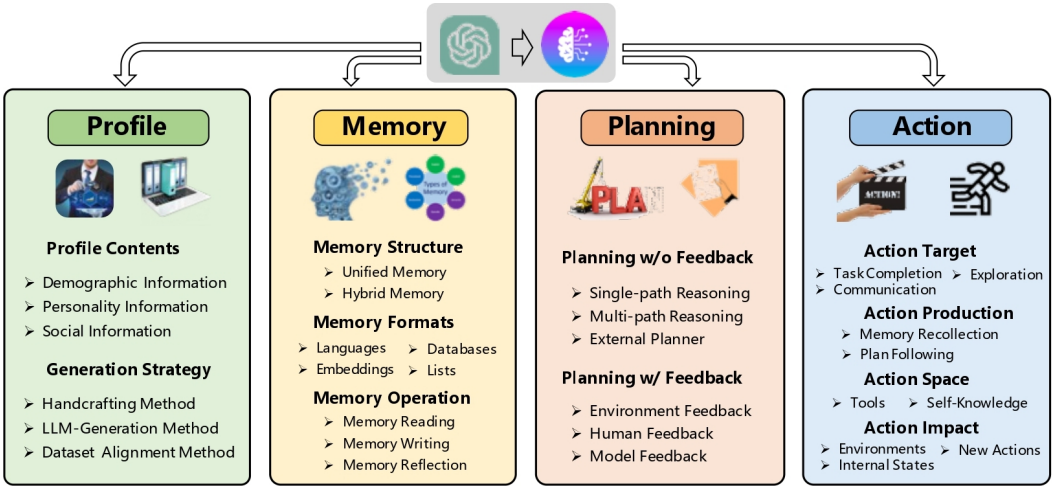


Figure 3.3: Conceptual framework for the design of LLM agents. Figure extracted from Wang et al. [39].

roles, thereby avoiding simple language outputs. For example, the Architect agent's output includes a system interface design and a sequence flow diagram.

Additionally, MetaGPT employs an indirect communication method, resembling stigmergy more than direct communication. MetaGPT uses a publish-subscribe mechanism, whereby agents publish their structured outputs to a message pool once they are ready. Agents must actively pull from the message pool to check if the information they need is available. This approach reduces inefficiencies, ensuring that information is never lost and does not need to be retransmitted, unlike direct communication protocols where information might be duplicated, recomputed, or completely forgotten.

To evaluate the efficacy of their framework, the authors conduct two types of tests. The first is benchmark-based, revealing that their framework surpasses the performance of simple Chain of Thought prompting [37] in both the HumanEval [5] and MBPP datasets [38]. Additionally, they develop their own SoftwareDev dataset consisting of 70 software engineering tasks. These tasks are more challenging than the previous datasets and require creating complete software products, such as simple games like the 2048 game, Flappy Bird, or Snake. The results show that their framework outperforms many agent-based and multi-agent baselines, including AutoGPT [31] and AgentVerse [35].

3.4 Conceptual Integration of Current Multi-Agent LLM Frameworks

Due to the surge in the publication of agentic and multi-agent LLM frameworks, several surveys in the field have been developed in the past year. These surveys aim to integrate all the different frameworks by identifying several principles or conceptual challenges each of these new frameworks is trying to address. Figure 3.3 summarizes the main modules that appear in most of these surveys. In the following, we describe a unified framework combining the different concepts covered in the surveys [40, 41, 39, 42]:

- **Profiling**: Profiling refers to the process of identifying and specifying the roles of the different agents, which influences the effectiveness and interactions between agents. Agents take on roles that describe their characteristics, capabilities, behaviors, and constraints. These roles are essential for ensuring effective collaboration and special-

ization within the multi-agent system. Profiling can be conceptualized through three different methods: pre-defined or handcrafted, model-generated, and data-derived. In the handcrafting method, profiles are manually created to fit specific needs, specifying a concrete prompt. In the LLM generation method, the LLM itself generates a profile for another agent. It is even possible to envision a profiling agent whose objective is to specify the prompts that define the profiles of the other agents. The data-derived method involves creating different LLM profiles based on existing data or datasets.

- **Planning**: Planning is broken down into two different components: plan formulation and plan reflection. Plan formulation refers to the idea of decomposing the goal and searching for the best plan to accomplish the task. A wide range of techniques have been developed to this end, including Chain of Thought [37], Tree of Thought [43], Graph of Thoughts [44], or external planners such as LLM+P [45]. Plan reflection refers to the idea of dynamically modifying the plan to adapt it by integrating feedback. The feedback can come from various sources, such as the environment (see React [46] and Voyager [47]), from humans (see Inner Monologue [48]), or from the model itself, including methods like Self-Refine [49] and Reflexion [50].

- **Memory**: Agents need memories to act effectively. LLMs possess three different types of memory. First, training memory includes the knowledge the model learns during pretraining, encompassing world knowledge, relational, common sense, semantic, and syntactic knowledge. Second, short-term memory refers to the information the models can store in their context length and use thanks to in-context learning. Third, agents can be provided with an external long-term memory by allowing them to interact with different data structures, such as vector databases. However, challenges are associated with this last type of memory. The LLM needs a criterion for relevancy to decide which information to read. Additionally, the agent needs to determine what to write to external storage, including merging similar information and removing outdated information. Finally, for the memory to be useful, the agent needs the ability to summarize different memories to infer insights that can be beneficial for the task at hand.

- **Environment and Actions**: The environment in which agents are embedded determines the actions they can take. Environments where LLM agents can act include physical environments, simulation environments, or virtual environments, determining the action space. To take actions, agents can rely on their internal knowledge, including their planning capability (see Voyager [47]), conversational capability (see ChatDev [51]), and common sense capabilities to understand or simulate aspects of human life (see Generative Agent [52]). Additionally, agents execute actions that trigger external tools, such as APIs (see HuggingGPT [53] and ToolFormer [54]), interact with different databases or knowledge bases, or even use external models (see MMReact [55] and ViperGPT [56]).

Additionally, Guo et al. [42], describe **communication**, one of the few aspects exclusive to multi-agent paradigms. They conceptualize communication along three different dimensions. The first dimension reflects the different communication paradigms, which include cooperative, competitive, or debate. The second dimension is the communication structure, which can be layered, centralized, decentralized, or a shared message pool. Stigmergy could also be a viable communication structure for multi-agent LLMs. Finally, there is the communication content dimension, which is usually text but can also include code, strategies, plans, and more.

Händler additionally introduces a crucial conceptual tradeoff between autonomy and alignment when designing multi-agent LLMs [34]. His analysis and taxonomy pay close attention

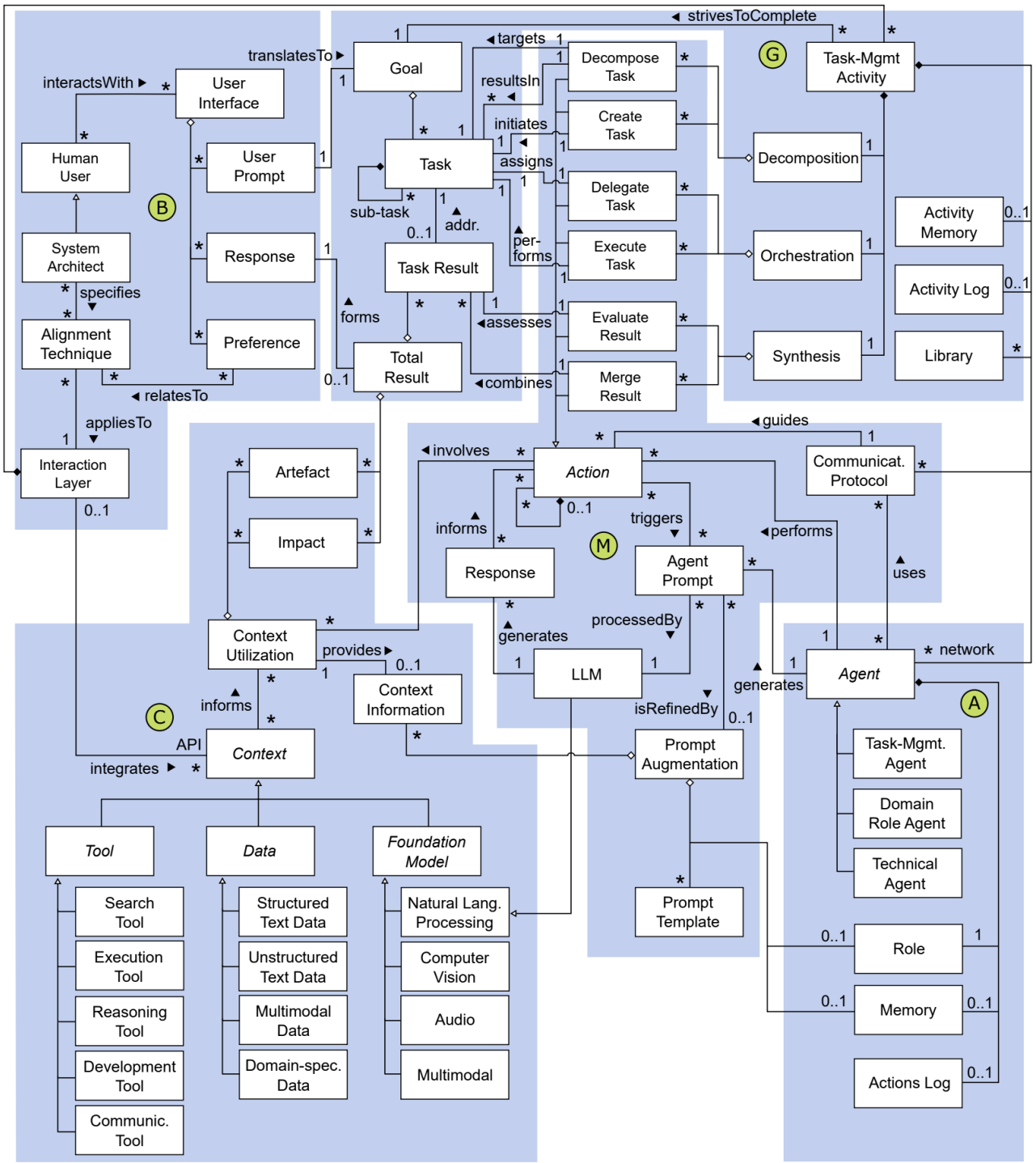


Figure 3.4: Domain ontology of the field of multi-agent LLMs represented as a UML diagram. Figure extracted from Händler [34].

to the intrinsically multi-agent mechanisms, or what he terms the *interaction layer*. Specifically, he develops a taxonomy of current multi-agent LLMs along three dimensions: autonomy, alignment, and architectural viewpoints, each divided into various aspects. Furthermore, he presents a comprehensive domain ontology of the different architectural concepts and concept relations found in multi-agent LLMs. The domain ontology is shown in Figure 3.4 and is divided into thematic blocks, including:

- **Concepts of Goal-Driven Task Management**: The initial prompt undergoes a process of decomposition, orchestration, and synthesis. Subtasks are connected either sequentially or in a graph-like manner.
- **Concepts of LLM Powered Agents**: There are different agent types. Task management agents include task creation, task prioritization, and task execution agents. Domain role agents are domain-specific experts. Technical agents are tech-specialized agents, such as Python or SQL agents.
- **Concepts of Multi-Agent Collaboration**: Various actions can be performed: DecomposeTask, CreateTask, DelegateTask, ExecuteTask, EvaluateResult, and MergeResult. Additionally, an action might involve Context Utilization or Prompt Augmentation. Agents communicate with each other using strategies categorized into strict finite processes (predefined action sequences), dialogue cycles, or multi-cycle process frameworks (the most flexible).
- **Concepts of Context Interaction**: These include tools (such as search, reasoning, and communication tools), data types (structured, unstructured, multimodal, and domain-specific data), and other specialized foundational models.
- **Concepts of Balancing Autonomy and Alignment**: Alignment manifests through the design choices of the architect in the interaction layer and through user prompts and instructions. Autonomy refers to the agent's ability to self-organize to fulfill a plan, encompassing all aspects and ideas of multi-agent collaboration, which can be fully self-driven and autonomous.

Chapter 4

Multi Agent LLM Architectures for Mathematics and Coding Problems

The intent of this chapter is to provide an overview of the multi-agent frameworks developed for solving coding and mathematics problems. We begin by reviewing the two foundational multi-agent frameworks on which our work is based: the Multi-Agent System for Conditional Mining (MACM) [2] and AgentCoder [3]. Following this, we present an overview of our enhanced AgentCoder+ framework and the newly developed meta-architecture. The empirical evaluation of our methods will be addressed in the subsequent chapter.

4.1 Multi-Agent System for Conditional Mining Overview

The Multi-Agent System for Conditional Mining (MACM) is a LLM-based multi-agent framework designed to solve mathematical problems. It was first introduced by Lei et al. in [2].

While current LLMs have demonstrated strong problem-solving abilities across a wide range of domains, they are known to struggle particularly with logical and mathematical reasoning. Although various prompt engineering techniques, such as Chain of Thought (CoT) [37], Tree of Thought [43], and Graph of Thoughts (GoT) [44], have been developed, they have not significantly enhanced the logical and mathematical problem-solving abilities of LLMs.

The MACM framework [2] aims to improve both the generalizability of prompt engineering methods and the problem-solving abilities of LLMs for complex mathematical reasoning. To achieve this, MACM introduces the concepts of Conditions and Objectives. Given a mathematical problem, the system first identifies a set of objectives to achieve and iteratively updates a set of conditions, which can be thought of as lemmas or propositions, until the final answer is reached.

At an abstract level, the system consists of three different agents: a thinker, a judge, and an executor. The thinker is responsible for generating ideas or thoughts, which iteratively build towards the solution of the mathematical problem. The judge's role is to validate these ideas or thoughts, determining their correctness and deciding when the solution is ready to be passed to the executor. The executor then translates the verified ideas into actual computations to arrive at the final answer, relying on a code interpreter since LLMs are not well-suited for performing mathematical computations directly.

Although the logical description of the system includes only three agents, the concrete implementation layer comprises six different agents, with two concrete agents corresponding

to each abstract agent. Figure 4.1 illustrates the six different concrete agents.

Specifically, the thinker has two main tasks. First, given the math problem, it extracts the primary objective or goal of the problem and generates a set of conditions to verify, referred to as the *objective generator* agent. Second, it generates new conditions for the judge to verify at each iteration of the algorithm, referred to as the *condition generator* agent.

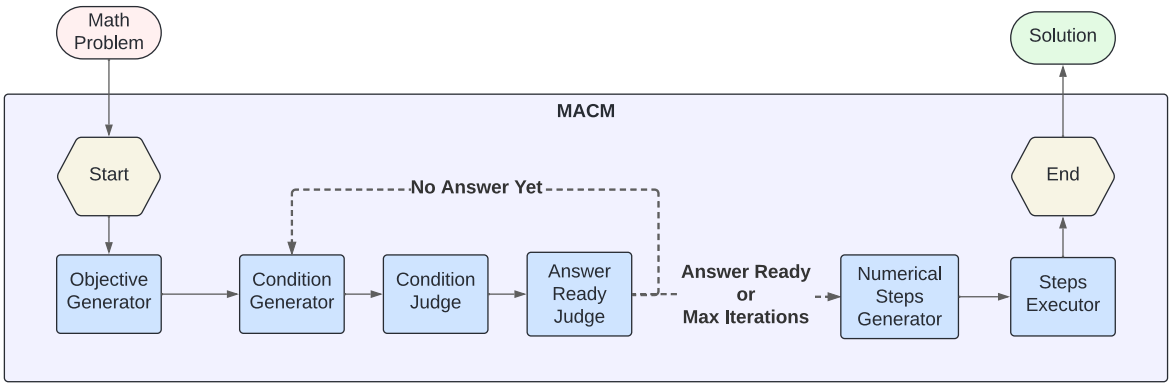


Figure 4.1: MACM implementation architecture.

The judge's contributions to the system are also twofold. First, the judge determines whether the condition generated by the thinker is correct; if not, it discards the condition. This is illustrated in Figure 4.1 as the *condition judge*. After verifying a new condition from the thinker, the judge assesses whether the generated conditions or lemmas provide a complete path to the answer. If they do, the executor generates the final solution; if not, the thinker is prompted to generate additional conditions, unless a maximum number of iterations has been reached. This is illustrated in Figure 4.1 as the *answer ready judge* agent.

Finally, the executor is responsible for using the generated conditions to compute the final solution. This involves two steps. First, the executor decides on the steps and calculations needed to arrive at the solution, performed by the *numerical steps generator* agent. Second, the predefined steps are executed by the *steps executor* agent, which has access to a code environment to reliably compute the math. This process creates a two-step Chain of Thought [37], where the executor first plans the steps without executing them and then executes the previously planned steps in a subsequent prompt.

By leveraging the concepts of conditions and objectives, the MACM framework abstracts away the specific details of various types of mathematical problems, thereby maximizing the applicability and generality of this problem-solving approach. In essence, it mimics the human approach to solving mathematical problems, where one iteratively generates and evaluates ideas until a solution is found.

The authors demonstrate that this framework improves the performance of several prompting strategies, including Chain of Thought (CoT) [37], Tree of Thought [43], and Graph of Thoughts (GoT) [44], across various benchmarks, such as the MATH dataset [4], the game of 24 points, and the sequence sorting problem.

4.2 AgentCoder Overview

The AgentCoder architecture is an LLM-based multi-agent framework designed to solve coding problems. It was first introduced by Huang et al. in [3].

Coding has emerged as one of the most significant applications of LLMs, given the vast

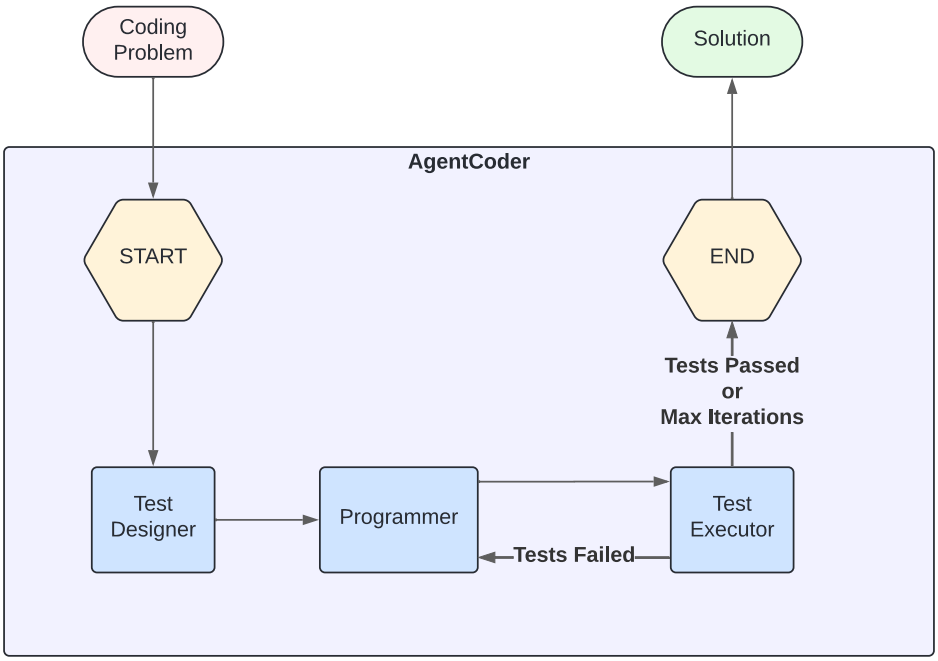


Figure 4.2: AgentCoder architecture.

amounts of open-source code available on the Internet. LLMs have developed strong coding skills, but their tendency to hallucinate remains a major issue, hindering widespread adoption. The goal of AgentCoder is to provide a multi-agent LLM framework that minimizes the impact of this propensity.

While other approaches, such as MetaGPT [36] and ChatDev [51], have also attempted to address these challenges, their solutions often lack an effective feedback mechanism and typically require a large number of agents. In contrast, AgentCoder offers a minimal LLM-based multi-agent architecture for solving coding challenges, resulting in a smaller token overhead compared to the alternatives.

AgentCoder is composed of three agents: the *test designer*, the *programmer*, and the *test executor*. The architecture is illustrated in Figure 4.2. First, the coding problem is given to the *test designer*, who generates a set of tests. Next, the coding problem is provided to the *programmer* agent. Once the programmer generates some code, the *test executor* runs the generated code against the tests. If the code passes all the tests, it is outputted. If the code fails any tests, feedback is generated on the failing test cases, and this feedback is provided to the programmer agent along with the original code for refinement. The refined code is then provided again to the test executor, and the process is repeated until the refined code passes the test cases or a maximum number of iterations is reached.

The test designer agent is prompted to generate a variety of test cases, including basic test cases, edge cases, and test cases for large-scale inputs. A key characteristic of the AgentCoder framework is that the test designer generates the tests independently of the code generation. This decision was made based on previous research suggesting a tradeoff between code generation performance and test generation accuracy when the same agent is used for both tasks [3, 57, 35].

The programmer agent is guided by a Chain of Thought [37] style prompt, designed to mimic the human programming process. The prompt instructs the agent to perform four actions sequentially: first, to understand and clarify the requirements of the problem; next, to think

about an approach or algorithm to solve the problem; then, to provide pseudocode for the solution; and finally, to generate the final code.

The executor agent is unique in that it is not LLM-based; it is simply a Python script capable of executing the code generated by the programmer. The executor's role in the AgentCoder framework is crucial, as it determines when the framework's execution will conclude. Specifically, if the code generates any errors or exceptions, feedback is provided to the programmer to refine the code. Conversely, if the code passes all the tests or the maximum number of iterations is reached, the code generated by the programmer in the last iteration is outputted as the final result.

Through a wide range of evaluation benchmarks, including HumanEval [5] and MBPP [38], AgentCoder consistently outperforms all the baselines, including the simple Chain of Thought [37], MetaGPT [36], and AgentVerse [35], demonstrating the great potential of multi-agent architectures to enhance LLM capabilities.

4.3 LangChain and LangGraph Overview

With the emergence of LLMs, the field of multi-agent LLM frameworks has experienced significant growth. This rapid expansion created a growing need for frameworks designed to facilitate the development of LLM-based applications and multi-agent systems.

LangChain and LangGraph were developed precisely to meet these needs. Throughout our development process, we relied heavily on these two programming frameworks to streamline the creation of our LLM-based multi-agent architectures. This section provides a brief introduction to these frameworks and how they were utilized in our experiments.

4.3.1 LangChain

LangChain is an open-source Python framework designed for developing LLM-based applications. Its primary goal is to create an abstraction layer over the LLM interface APIs of various providers. This standardizes the way agentic LLM frameworks interact with model providers, improving interoperability across different LLM models.

One of the most valuable features of LangChain is its ability to abstract away the process of obtaining parsed outputs from the model. For instance, if we have a tester agent in our framework, we may want to extract both a numerical value representing the percentage of tests passed and detailed feedback on any errors or exceptions produced by the code. Since LLMs typically provide only textual outputs, parsing these specific fields from the text can be challenging. Fortunately, LangChain offers functionality to simplify this process, making development more efficient.

Additionally, one of the key concepts in LangChain is the *chain*. Chains allow developers to combine multiple calls to LLMs, output parsers, and even tools into a single *runnable* item. This composability is highly beneficial for constructing complex agents and simplifies the overall development process.

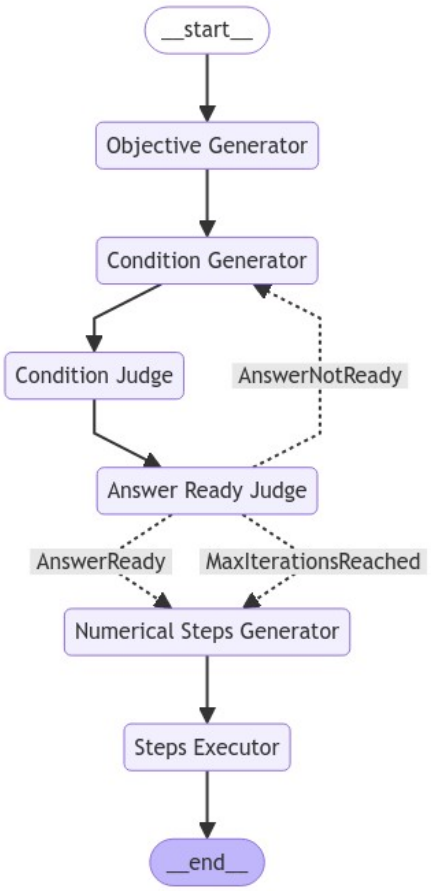
4.3.2 LangGraph

LangGraph is an open-source Python extension of the LangChain framework, designed specifically for the development of multi-agent LLM-based applications. It is built on technologies such as Apache Beam, Pregel, and NetworkX.

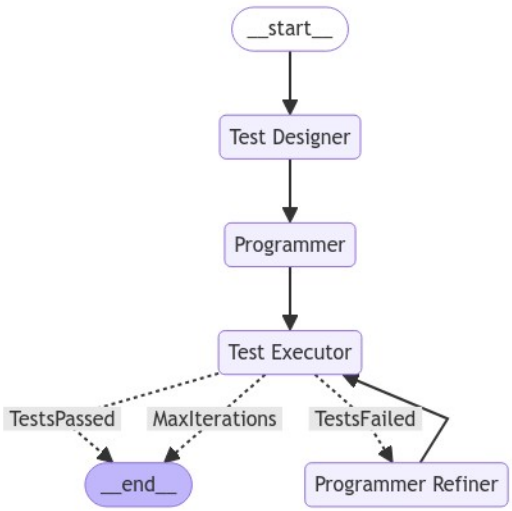
LangGraph provides several highly useful features. First, it enables the creation of graph-like structures that mimic the execution flow of a multi-agent LLM application. It supports cycles and branching, allowing multi-agent systems to implement loops and conditionals within the framework. Additionally, it offers persistence by saving the state of the graph at every step of execution.

Perhaps the most powerful feature of LangGraph is its ability to easily compose multiple multi-agent LLM frameworks into a single cohesive system, a feature that we will explore in subsequent sections. This capability was instrumental in enabling a clear and elegant implementation of our meta-framework.

We implemented the original MACM and AgentCoder frameworks discussed in previous sections using LangGraph. The execution graphs of these implementations can be seen in Figure 4.3, closely resembling Figures 4.2 and 4.1. Our implementations using the LangChain and LangGraph libraries resulted in cleaner, more elegant code compared to the original implementations and allowed us to seamlessly compose them, as will be demonstrated in the following sections.



(a) MACM framework.



(b) AgentCoder Framework.

Figure 4.3: Visualization of the implementations of the MACM and AgentCoder frameworks using LangGraph. Note that for the AgentCoder implementation the *programmer* agent has been separated into the *programmer* and *programmer refiner* agents for responsability separation and clarity.

4.4 AgentCoder+

While the AgentCoder architecture significantly improves performance compared to previous prompting mechanisms and multi-agent frameworks, there remains room for further enhancements.

One key limitation is that, although there is a feedback and refinement mechanism for improving the code generated by the programmer agent, the test cases are generated only once. Through empirical observations, we found that many of the failed test cases in AgentCoder contained errors during the test design phase, which hindered the code refinement process of the programmer agent, as the generated test cases were immutable.

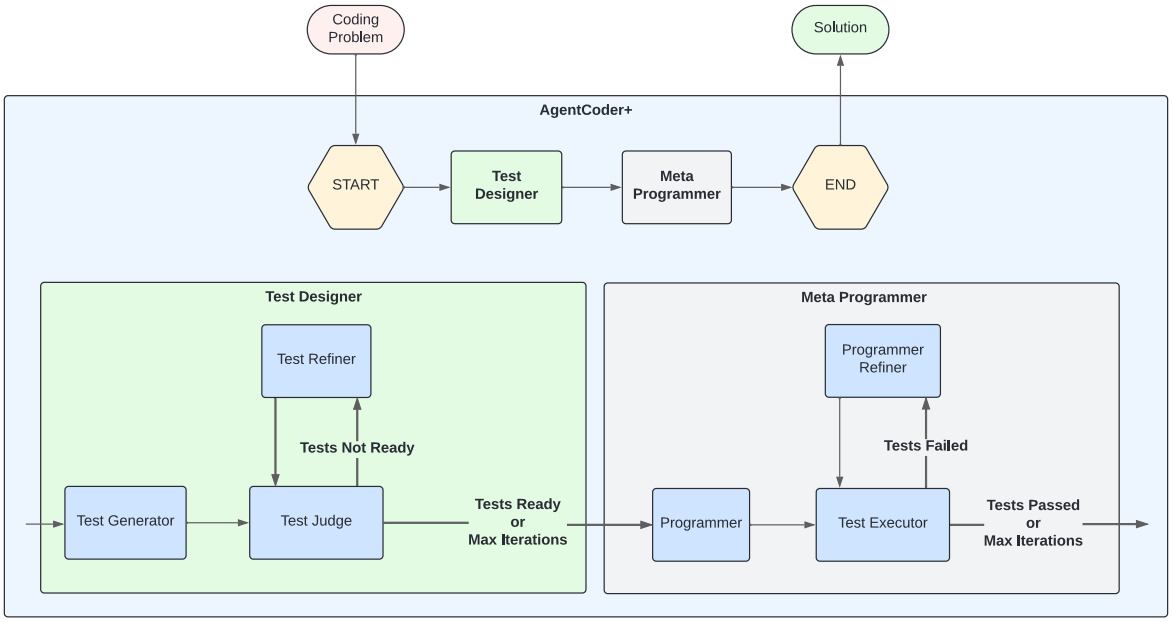


Figure 4.4: AgentCoder+ architecture.

To address this issue, we introduce AgentCoder+, illustrated in Figure 4.4, which conceptualizes the code generation process into two independent modules or meta-agents: test case generation (Test Designer) and code generation (Meta Programmer). The code generation phase remains unchanged from the AgentCoder architecture, with an initial code generation process that is iteratively refined to pass all the test cases.

The primary difference lies in the test generation step. Instead of generating the test cases only once—which often leads to erroneous test cases—we introduce a test judge agent whose role is to assess the correctness of the generated test cases. If the test judge detects errors in any of the test cases, they are sent to a test refiner agent for improvement. This loop is repeated until the tests are approved by the test judge or a maximum number of iterations is reached.

Once these improved test cases are ready, the test executor in the code generation phase will have a more reliable set of tests against which to evaluate the generated code. This results in more accurate feedback and ultimately improves the quality of the final code produced by the framework.

Figure 4.5 shows the actual implementation of the framework using LangChain, closely resembling Figure 4.4. It is worth noting how the composability of two simple langgraphs (Test Designer and Meta Programmer) is greatly facilitated by the LangGraph framework. The prompts for the newly implemented agents can be found in Appendix Section A.1.

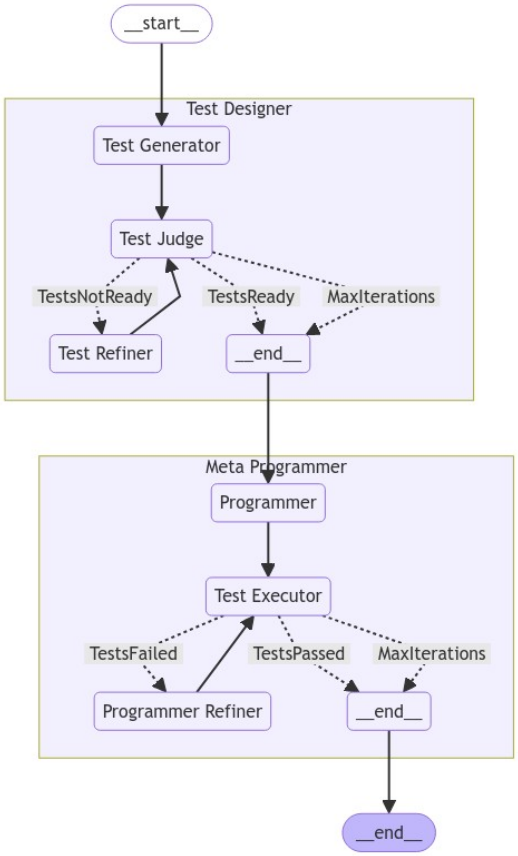


Figure 4.5: Visualization of the implementation of the AgentCoder+ framework using LangGraph.

4.5 Generalized Meta Architecture

The goal of this section is to introduce a generalized meta-architecture for both mathematical and coding problems. This meta-architecture is designed to be as flexible as possible, so it can be applied to various prompting mechanisms or multi-agent frameworks. We will first provide an overview of the meta-architecture, followed by its application to the MACM and AgentCoder frameworks.

4.5.1 Meta-Architecture Overview

Since the emergence of LLMs, there has been growing interest in the concept of adaptive compute. LLMs, in their current form, use roughly the same amount of compute for every query. However, humans tend to allocate more time and mental resources to solving complex problems compared to simpler ones. For instance, a mathematician will likely spend much more time solving a complex integral than solving a basic arithmetic problem.

Similarly, we would expect advanced AI models like LLMs to adapt their compute based on the complexity of the problem they are tasked with solving.

With this in mind, we introduce a meta-architecture for both coding and math problems, aimed at enhancing the problem-solving abilities of any prompting mechanism or multi-agent framework. This meta-architecture achieves this by running multiple iterations and automatically selecting the best solution. The overall meta-architecture is depicted in Figure 4.6.

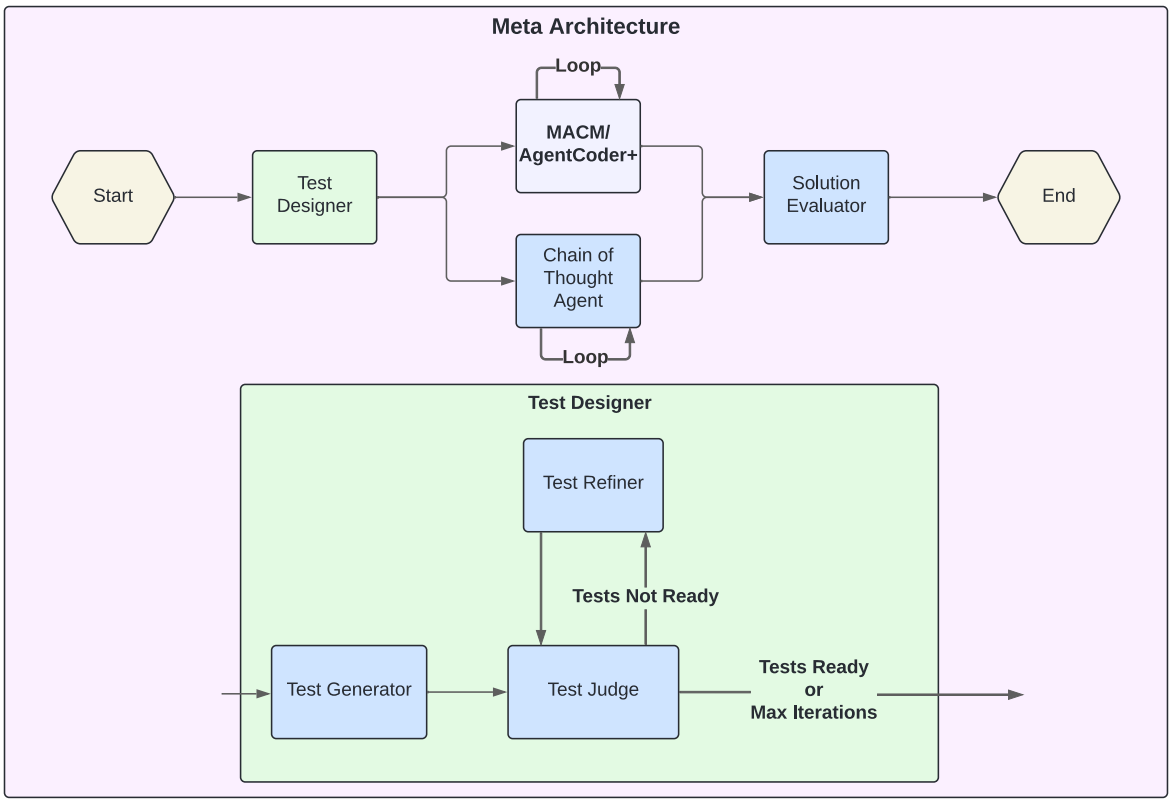


Figure 4.6: Meta architecture.

The process begins with the design of a set of tests that the solution to a given math or coding problem should satisfy. For math problems, these take the form of conditions or propositions that the answer must meet. In coding problems, they are test cases. These tests undergo a refinement process similar to that of AgentCoder+, until the test judge agent approves them.

Once the tests are ready, the selected framework, either MACM or AgentCoder+, is run multiple times on the same problem, with each generated solution being saved. In parallel, a single-agent Chain of Thought model also solves the problem multiple times. In the final step of the meta-architecture, the solution evaluator agent takes all the generated solutions and runs them against the predefined tests. The final solution is selected based on which one passes the highest number of tests or, in the case of a tie, the most frequently generated solution.

The number of solutions generated by both the Chain of Thought agent and the multi-agent frameworks are adjustable parameters, allowing for flexibility depending on the complexity of the problem.

Additionally, it is worth noting that the meta-architecture is highly flexible and could potentially be applied to any text-based problem-solving challenge where the solution can be tested for correctness. In the next two subsections, we demonstrate this generality by applying it to two different problem-solving tasks: mathematics and coding.

4.5.2 MetaMACM

When the MACM architecture is applied to the meta-architecture, we obtain MetaMACM. This architecture, along with its implementation using LangChain, is illustrated in Figure 4.7.

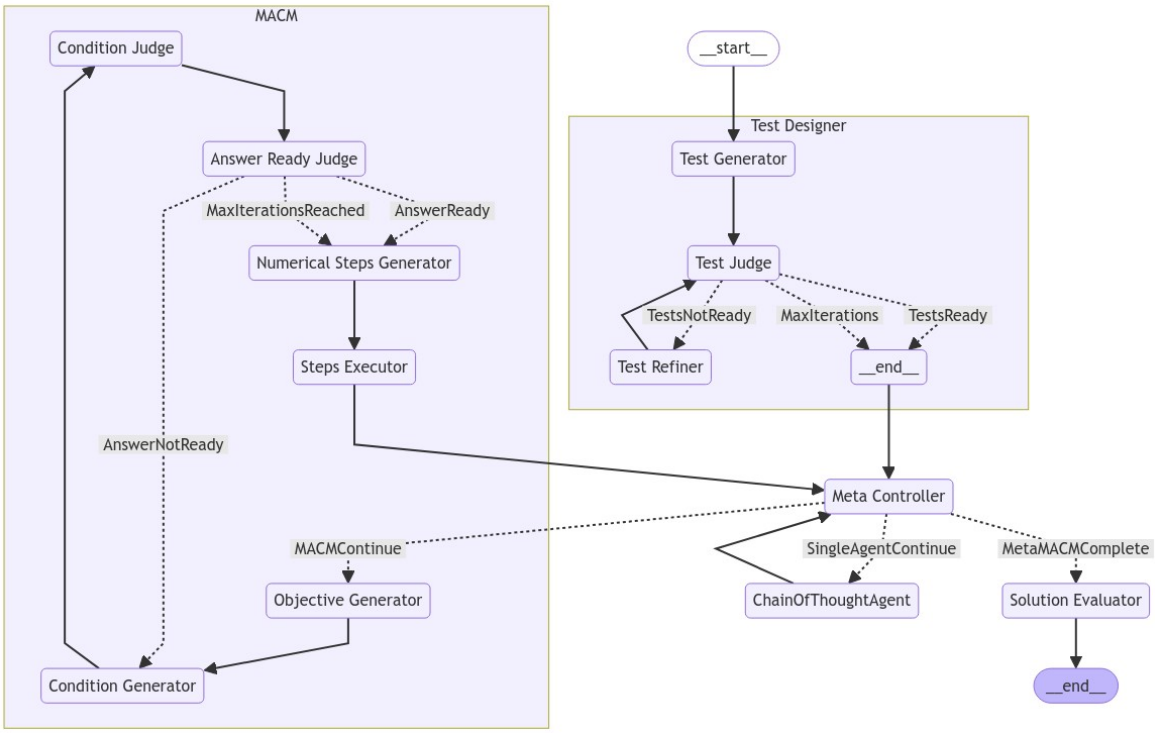


Figure 4.7: MetaMACM architecture implementation using LangChain.

In the LangChain implementation of MetaMACM, an additional agent, referred to as the *Meta Controller*, was introduced to manage the execution flow of the meta-architecture. However, this agent does not directly influence the problem-solving process itself; it solely acts as an execution flow controller, ensuring that the various agents within the framework are properly coordinated. Additionally, both the MACM and Chain of Thought agent loops are each executed three times in the final experiments, providing a total of six potential solutions for the *solution evaluator* agent.

The prompts for the newly implemented agents can be found in Appendix Section A.2.

4.5.3 MetaAgentCoder+

Applying the meta-architecture to the previously discussed AgentCoder+ multi-agent LLM framework results in a new architecture we term MetaAgentCoder+. This architecture, and its implementation using LangChain, is depicted in Figure 4.8.

Similar to MetaMACM, this architecture includes an additional *Meta Controller* agent to manage the framework's execution flow within LangGraph. Aside from this, the core structure of MetaAgentCoder+ consists of the *Meta Programmer* meta-agent, as the *Test Designer* is already integrated into the meta-framework with the same role. This setup allows test cases to be generated only once and reused across all AgentCoder+ iterations, significantly improving the efficiency of the framework.

Additionally, both the AgentCoder+ and Chain of Thought agent loops are each executed three times in the final experiments, providing a total of six potential solutions for the *solution evaluator* agent.

The prompts for the newly implemented *solution evaluator* agent can be found in Appendix Section A.3.

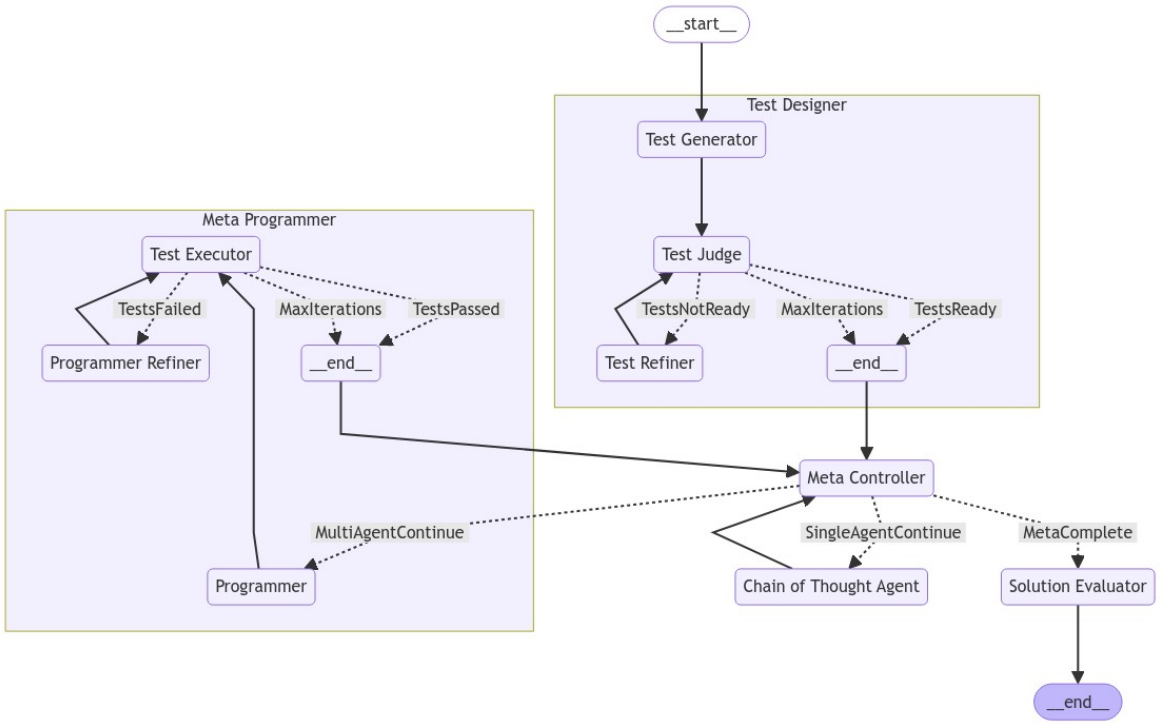


Figure 4.8: MetaAgentCoder+ architecture implementation using LangChain.

Chapter 5

Experimental Evaluation

This chapter explores the experimental results of our newly developed multi-agent LLM frameworks. To this end, we begin with a brief overview of the benchmarks used for evaluation: MATH [4] and HumanEval [5]. We then proceed to test the performance of our frameworks using GPT-4o and GPT-4o-mini [25] as the underlying LLMs.

5.1 Benchmarks

In this section, we provide a brief overview of the two primary benchmarks used throughout the experimental evaluation of our multi-agent frameworks: the MATH benchmark [4] and the HumanEval benchmark [5].

5.1.1 MATH Benchmark

The MATH benchmark was introduced by Hendrycks et al. [4] to evaluate the mathematical reasoning abilities of AI systems.

This benchmark includes a dataset of 12,500 mathematics problems written in LaTeX, each accompanied by a canonical solution. The problems are categorized into seven distinct topics: Prealgebra, Precalculus, Counting and Probability, Intermediate Algebra, Algebra, Number Theory, and Geometry.

Additionally, each problem in the MATH dataset is assigned a difficulty level, ranging from 1 (the simplest) to 5 (the most difficult). Table 5.1 provides three randomly sampled examples from the MATH dataset along with their respective solutions.

Notably, at the time of the benchmark's release in 2021, Hendrycks et al. [4] evaluated several models, including GPT-2 [22] and GPT-3 [23], achieving minimal success with performance scores between 3% and 7%. The authors argued that further theoretical advances, beyond mere model scaling, were necessary to improve AI's mathematical reasoning abilities. However, as model sizes have continued to grow, the performance of these models on mathematical problems has significantly improved.

5.1.2 HumanEval Benchmark

The HumanEval benchmark, introduced by Chen et al. [5], is designed to evaluate the coding abilities of AI systems.

Problem: How many (non-congruent) isosceles triangles exist which have a perimeter of 10 and integer side lengths? **Solution:** Let x be the measure of each of the equal sides. Since the perimeter is 10 units, the side lengths measure x, x, and $10-2x$ units. Since the third side length must be positive, we have $10-2x>0$ which implies $x<5$. By the triangle inequality, the sum of the two equal sides must exceed the third side. Solving $x+x>10-2x$ gives $x>2.5$. There are $\boxed{2}$ integers strictly between 2.5 and 5.
Problem: Find a quadratic with real coefficients and quadratic term x^2 that has $5-4i$ as a root. **Solution:** Since the root $5-4i$ is nonreal but the coefficients of the quadratic are real, the roots must form a conjugate pair. Therefore, the other root is $\overline{5-4i}=5+4i$. To find the quadratic, we can note that the sum of the roots is $5-4i+5+4i=10$ and the product is $(5-4i)(5+4i)=25+16=41$. Then by Vieta's formulas, we know that the quadratic $\boxed{x^2-10x+41}$ has $5-4i$ as a root.
Problem: On a survey, students must give exactly one of the answers provided to each of these three questions: • a) Were you born before 1990? (Yes / No) • b) What is your favorite color? (Red / Green / Blue / Other) • c) Do you play a musical instrument? (Yes / No) How many different answer combinations are possible? **Solution:** The first question has 2 answer choices, the second 4 answer choices, and the third 2. So there are $2\times4\times2=\boxed{16}$ different answer sets.

Table 5.1: Examples of problems from the MATH benchmark along with their canonical solutions.

The benchmark consists of a dataset of 164 coding problems, where each problem provides a method description, and the AI system is expected to generate the corresponding code that implements the method. Two examples from the dataset are provided in Table 5.2.

To measure the accuracy of the AI system, Chen et al. [5] propose using the pass@k metric. The idea behind pass@k is to generate k samples using the model, and the problem is considered successfully solved if any of the k samples are correct. However, simply generating k samples can result in high variance, so to compute the pass@k metric, typically $n \geq k$ samples are drawn, and the pass@k score is computed using the unbiased estimator

$$\text{pass@k} = \mathbb{E}_{\text{problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right],$$

where c represents the number of correct samples.

In our case, since we generate only one sample per problem, we will report the pass@1 metric, which approximates the accuracy of the model when only a single solution is generated.

To evaluate the correctness of a given potential solution, the benchmark provides a set of hidden test cases for each coding problem. Any correctly implemented method must pass these test cases. We utilized these hidden tests to assess the validity of the generated solutions.

5.2 Results

This section presents the experimental results of the multi-agent LLM frameworks developed in the previous chapter, using the benchmarks discussed earlier. We begin by briefly

Prompt:

```
        def unique(l: list):
            """Return sorted unique elements in a list
            >>> unique([5, 3, 5, 2, 3, 3, 9, 0, 123])
            [0, 2, 3, 5, 9, 123]"""
```

Canonical Solution:

```
            return sorted(list(set(l)))
```

Prompt:

```
def cycpattern_check(a , b):
    """You are given 2 words. You need to return True if the second word or
    any of its rotations is a substring in the first word
    cycpattern_check("abcd","abd") => False
    cycpattern_check("hello","ell") => True
    cycpattern_check("whassup","psus") => False
    cycpattern_check("abab","baa") => True
    cycpattern_check("efef","eeff") => False
    cycpattern_check("himenss","simen") => True """
```

Canonical Solution:

```
    l = len(b)
    pat = b + b
    for i in range(len(a) - l + 1):
        for j in range(l + 1):
            if a[i:i+l] == pat[j:j+l]:
                return True
    return False
```

Table 5.2: Two examples from the HumanEval Dataset.

reviewing our initial experiments on book summarization before moving on to the results of AgentCoder+, MetaMACM, and MetaAgentCoder+ on the MATH and HumanEval benchmarks.

5.2.1 Book Summarization

Our first set of experiments focused on book summarization, as we initially believed it would be an ideal testbed to evaluate the potential of multi-agent LLM architectures. Most current LLMs cannot fit entire books within their context window, making book summarization a promising task for an LLM-based multi-agent system. A hierarchical system could decompose a book into chapters and sections, summarize them separately, refine the summaries to

Model	Chain of Thought	MACM Framework	MetaMACM Framework
GPT-4o Mini	54.29	55.24	56.19
GPT-4o	57.14	59.05	61.19

Table 5.3: Results for GPT4o and GPT4o Mini in the MATH benchmark using different architectures. Results provided as overall accuracy (%).

minimize errors, and then merge them into a cohesive final summary.

To test this idea, we utilized BooookScore, developed by Chang et al. [58]. BooookScore is an automatic metric designed to evaluate the quality of book summaries based on a taxonomy of eight common types of errors that LLMs often make. The metric works by assessing the proportion of sentences in the LLM-generated summaries that contain one of these identified errors. To determine whether a sentence contains an error, the system prompts an LLM to identify any of the errors in the taxonomy. The final BooookScore is the proportion of sentences in the summary that do not contain errors, with a perfect score of 1 indicating that no errors were found.

Intrigued by this automatic approach to measuring the quality of book summaries, we tested the performance of the hierarchical merging method proposed by the authors. This method involves breaking the book into fixed-length chunks, summarizing them individually, and then progressively combining these summaries until a final summary is produced.

To our surprise, when testing the latest OpenAI model, GPT-4o, on several freely available books from Project Gutenberg, we found that the benchmark was effectively saturated, with many summaries achieving the maximum score of 1. As a result, we shifted our focus to more challenging tasks, specifically mathematics and coding problems.

5.2.2 MetaMACM Results

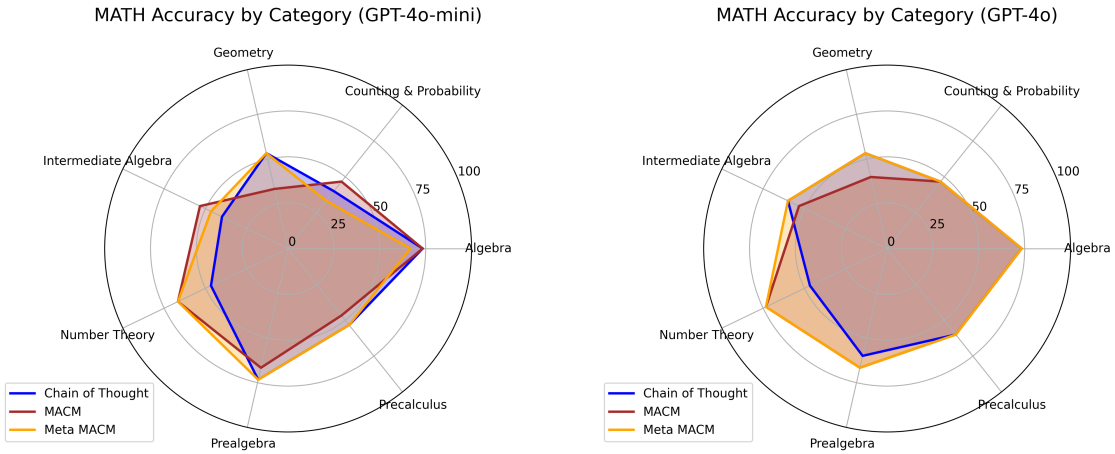
To evaluate the performance of the MetaMACM framework, we used the MATH benchmark. For our tests, we randomly selected three math problems from each of the seven problem categories and each of the five difficulty levels, resulting in 15 problems per category and 105 problems in total.

Once these problems were selected, we compared the performance of our MetaMACM framework against two baselines: an individual Chain of Thought (CoT) [37] agent and the original MACM architecture. The tests were performed using two underlying LLM models: GPT-4o and GPT-4o mini. GPT-4o is OpenAI's most advanced model, while GPT-4o mini is a more economical version with slightly reduced cognitive capabilities.

The main results are presented in Table 5.3. For GPT-4o mini, the less capable model, we observed a slight improvement in accuracy when moving from the single-agent Chain of Thought prompting method to the MACM framework, followed by a similar improvement when transitioning from the basic MACM framework to our MetaMACM framework. In the more capable model, GPT-4o, the trend is similar, but the improvements are more pronounced.

We hypothesize that the greater improvement in accuracy for more capable models reflects their ability to generate more useful feedback and better tests compared to less capable models, which in turn accentuates the advantages of transitioning to a multi-agent framework like MetaMACM.

Additionally, Figure 5.1 provides a more detailed overview of the results, grouped by the



(a) GPT-4o Mini

(b) GPT-4o

Figure 5.1: Accuracy per category in the MATH benchmark for the different multi-agent LLM architectures.

seven different categories of math problems. For the less capable GPT-4o mini model, the advantage of moving from a single agent to MetaMACM is less consistent: while performance improves in categories such as Intermediate Algebra, Number Theory, and Precalculus, there are some areas, like Counting and Probability and Algebra, where performance slightly decreases.

In contrast, for the more capable GPT-4o model, the benefits of using MetaMACM are clear. Performance significantly improves in categories like Geometry, Number Theory, and Prealgebra, while remaining stable in the other categories.

Overall, these results demonstrate the effectiveness of the MetaMACM framework, as it consistently provides better results across the tested models, particularly in the more advanced GPT-4o model.

5.2.3 AgentCoder+ and MetaAgentCoder+ Results

To evaluate the performance of AgentCoder+ and MetaAgentCoder+, we used the HumanEval benchmark. For these experiments, we ran our baselines and frameworks on the entire HumanEval dataset, consisting of 164 problems, and reported the accuracy of each method. A solution was considered correct only if it passed all the hidden tests provided by the benchmark.

As in the MATH and MACM experiments, we compared our developed methods against two baselines: an individual Chain of Thought (CoT) [37] agent and the original AgentCoder architecture. Similarly, we used two models for our final results: GPT-4o and GPT-4o mini.

The results of these evaluations are presented in Table 5.4. For the less capable GPT-4o mini model, we observed mixed results. While both the AgentCoder and AgentCoder+ multi-agent architectures performed better than the individual Chain of Thought agent, AgentCoder+ showed a slight decrease in performance compared to AgentCoder. We attribute this degradation to erroneous feedback that the *test judge* agent might provide, likely due to the more limited cognitive capabilities of the GPT-4o mini model. However, when using the more capable GPT-4o model, AgentCoder+ improved performance over both the individual

Model	Chain of Thought	AgentCoder	AgentCoder+	MetaAgentCoder+
GPT-4o Mini	87.20	89.02	88.41	92.07
GPT-4o	90.24	91.46	92.07	93.29

Table 5.4: Results for GPT4o and GPT4o Mini in the HumanEval benchmark accross the different developed frameworks. Results provided as overall accuracy (%).

Chain of Thought agent and the original AgentCoder architecture.

Furthermore, the MetaAgentCoder+ architecture consistently outperformed all baselines and frameworks. Notably, the performance of MetaAgentCoder+ using the less capable GPT-4o mini model exceeded that of the individual Chain of Thought agent using the more powerful GPT-4o model, demonstrating that we can effectively trade off inference compute with training compute.

Chapter 6

Towards Large Language Model Organizations

While the literature review chapter focused on studying currently existing multi-agent LLM frameworks, this chapter is much more speculative in nature and aims to envision how this field of multi-agent LLM frameworks will or should evolve. It is our belief that the end goal of the multi-agent LLM field should be the creation of a Collective Intelligence where the intelligence of the collective is far greater than sum of the individuals.

We thus aim to step back and explore other fields of knowledge that have previously addressed the challenge of designing Collective Intelligence. In particular, we will turn our attention to the field of Multi-Agent System Organizations, which reached its zenith in the 2000s under the paradigm of agent-oriented software engineering. We argue that this often-overlooked sub-field of Multi-Agent Systems (MAS), which has already studied and tackled the problems of designing organizations of agents capable of achieving some form of Collective Intelligence, provides the missing conceptual grounding needed to achieve scalable Collective Intelligence with contemporary LLM systems—an element that was not available during the peak of this research direction.

6.1 The Need for LLM Organizations

While there has been remarkable progress in the development of multi-agent LLM systems, the field still faces several challenges that limit their widespread adoption. In principle, the potential of this field is vast and will continue to grow as the underlying *intelligence layer* provided by LLMs keeps improving. Ultimately, when the underlying intelligence provided by the *intelligence layer* is comparable to the general ability and intelligence of a human being—whether achieved by LLMs or another type of technology—the potential of this field is to create a *collective intelligence* whose capabilities far surpass those of any individual human. Just as human organizations or even humanity as a whole, seen as an individual superorganism, possess intelligence and problem-solving capacity that exceed those of any individual human, so too could a well-coordinated system of multi-agent LLMs.

Thus, the purpose of the field of multi-agent LLMs should be to harness as much as possible with current technologies the power of collective intelligence and to reflect on how future, more capable systems could further harness it. To explore this potential from a conceptual point of view is the intent of this chapter.

To harness the potential of *collective intelligence*, we have identified two main key issues that

need to be tackled, which current multi-agent LLM systems are overlooking. First is the need to shift the focus from individual agent capabilities to the creation of effective mechanisms at the *interaction layer*. As discussed in the literature review chapter, most current multi-agent LLM approaches focus mainly on the individual agent level, with key unifying concepts identified in most reviews being profiling, planning, memory, and environment and actions. Händler [34] was one of the first to shift the focus from the individual agent level to the *interaction layer*. We believe that focusing on the different coordination, cooperation, communication, hierarchical decomposition, supervision, competition, and other types of mechanisms at the *interaction layer* is key to developing much more capable multi-agent LLM systems.

Second is the challenge and potential of scaling the number of agents in the system, which has also been identified by Xi et al. [41] and Guo et al. [42]. Indeed, Xi et al. [41] propose different types of scaling, including predetermined scaling specified manually by the system designer, dynamic scaling, and autonomous scaling, where agents can instantiate other agents when needed, providing the system with a self-organization mechanism. However, current multi-agent LLM systems are not scalable, as the agents and their interactions are designed in a hard-coded manner by the system architect. We argue that shifting the focus from individual agents towards the interaction layer will provide a way to construct more scalable multi-agent LLM systems. Scaling the number of agents can enhance task specialization, allowing the system to tackle more complicated tasks.

To address these challenges and harness the power of *collective intelligence*, we argue that the well-established field of Multi-Agent Systems (MAS) provides the necessary conceptual tools. In particular, the subfield of Multi-Agent Organizations, which sits at the intersection of Organization Theory and MAS, has already tackled these kinds of problems successfully in the late 1990s and early 2000s. We now delve into this field to understand how it can help us develop much more capable multi-agent LLM systems.

6.2 From Agent-Centered to Organization-Centered MAS

In 2003, Ferber et al. [59] first introduced the concept of Organization-Centered MAS (OCMAS). Ferber argued that until then, the field of MAS had been centered around individual agents, which he termed Agent-Centered MAS (ACMAS). In ACMAS, the designer focuses on the local behaviors of the agents and their local interactions, without paying much attention to the overall architecture and structure of the system [60]. In ACMAS, the organization only exists as a phenomenon emerging from the local interactions of the agents and is not explicitly defined. The organization thus emerges in a self-organized manner from the local interactions of the agents, similar to the collective intelligences emerging in nature, such as ant colonies. However, this approach is often undesirable, as the emergent characteristics of the overall system are usually unpredictable and often counterproductive.

The alternative MAS viewpoint proposed by Ferber et al. [59] is the Organization-Centered MAS (OCMAS), in which the focus shifts towards the macro level, allowing for a clearer understanding and better design of the overall system structure. In this new approach, the MAS designer first designs the overall structure of the system at a macro level to best fit a given task or problem-solving paradigm, and the individual behaviors of the agents are programmed accordingly at a later stage. This represents a paradigm shift from the bottom-up approach implicit in ACMAS to a top-down approach represented by OCMAS.

OCMAS involves taking organizational concepts such as groups, structures, roles, and dependencies and giving them equal or greater ontological importance than the individual agents.

This can be key to building large-scale complex systems [60]. OCMAS also allows for limiting the scope of interactions among different agents by breaking down high-level complex tasks into smaller subtasks within a hierarchical organization, ensuring that no single agent needs to understand or be aware of the high-level task. The OCMAS approach thus incorporates insights from Organization Theory to structure MAS in a manner more akin to how humans work in organizations.

Following similar reasoning, we argue that the field of multi-agent LLMs needs a similar paradigm shift—from an agent-centric approach to an organization-centric approach that focuses on designing effective mechanisms to make the *interaction layer* as effective and scalable as possible. In the following section, we further explore some ideas from the field of multi-agent organizations, taking the OCMAS perspective, which are directly relevant to our aim of harnessing collective intelligence using LLMs.

6.3 MAS Organizations Frameworks to Enhance Multi-Agent LLMs

In this section, we briefly describe several frameworks from the field of Multi-Agent Organizations that contain useful concepts and ideas for the development of the field multi-agent LLMs.

Agent/Group/Roles (AGR) Framework.

Introduced by Ferber et al. [61] in 2003, this framework provides foundational concepts of agents, groups, and roles as key aspects of MAS. This work is also foundational in the paradigm shift from ACMAS to OCMAS.

The authors enumerate a series of drawbacks of ACMAS, most of which are still applicable to some of the most popular current multi-agent LLM frameworks. They define the ACMAS as a MAS where every agent can communicate with every other agent, it is the responsability of agents to constrain their relation with other agents, and agents are thought of as providing services which are available to all agents in the system. In such an scenario, it is argued that the patters and interactions among the agents are inherently unpredictable, as emergent and potentially undesirable behaviour might emerge. But the main weakness of such a system is its lack of modularity, as every agent is accessible from every other agent, which creates deep scalability problems [61].

On the contrary, they characterize the OCMAS approach to MAS as one in which the overall organization can be partitioned in groups, in which the behaviour of agents are functionally related with the overarching goal of the organization and there is a dynamic relationship between agents based on taks, protocols or roles which defines some supra-individuality [61]. The organizational level of the OCMAS is thus concerned with the relations between agents, but not so much on how agents behave themselves, and the ability to partition the organization into groups allows for modularization in the organization, providing each group with a different context for them to interact with one another.

To concretize their description of OCMAS and make it computationally implementable, Ferber et al. [61] introduced the Agent/Group/Role (AGR) framework. A UML description of this meta-framework is provided in Figure 6.1. Specifically, an agent is an entity that adopts roles within groups, with no restrictions placed on how the agent functions, only on its relationships with other system components, thus adhering to the OCMAS design pattern. A group is composed of a set of agents sharing certain characteristics, allowing for the partitioning of the organization, which in turn encourages modularity and enhances scalability. Finally, roles describe an agent's function within a group.

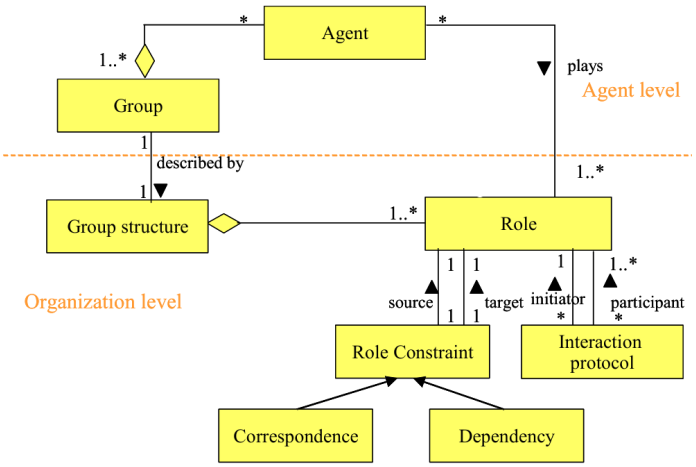


Figure 6.1: AGR meta-model UML diagram. Figure extracted from Ferber et al. [61].

While simple, these three concepts formed the conceptual basis for the development of subsequent, more elaborate frameworks for OCMAS. Additionally, this framework highlights how MAS organizations can inform the development of multi-agent LLMs. While the concepts of agents and roles are already well-integrated into the multi-agent LLM literature, the concept of groups within an organization to enhance modularity and scalability remains underdeveloped.

MOISE and MOISE+.

The Model of Organization for multI-agent SystEms (MOISE) is an organizational modeling language introduced by Hannoun et al. [62] with the aim of reconciling both the ACMAS and OCMAS approaches to MAS.

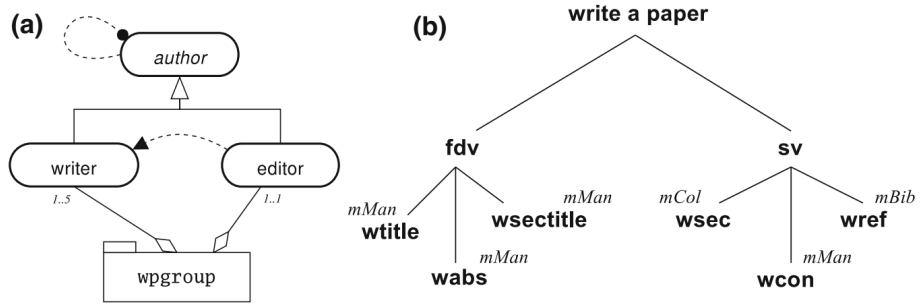
To achieve this, they begin by separating the organizational structure (OS) from the organizational entity (OE). The concept of the OS is very similar to the organizational level in the AGR framework. It refers to the set of roles, groups, links, and rules that define the overall structure of the organization, without specifying the agents and their capabilities within the system. Conversely, the OE refers to the actual set of agents operating under the OS, thus representing the instantiation of a particular OS within a group of agents.

Roles, groups, and links are the fundamental concepts that structure both the OS and the OE in MOISE. While roles constrain the behaviors of agents in the MAS, organizational links define the structure of interactions between agents. These links are categorized as either communication links (which define the communication patterns of agents in the system), authority links (which represent the subordination of one agent to another concerning a task or mission), and acquaintance links (which represent the set of agents or groups that an agent can interact with).

Groups represent how the OS is realized into an actual OE. A group consists of a set of roles, a set of missions, and a set of links related to the roles within the group. An OS is composed of a set of roles, links, and groups, while an OE consists of one or more OS, a set of agents each associated with a particular role, and a set of instantiated links and groups.

Hübner et al. [63] argue that organizational models often focus on only one of three dimensions of organizations: the structural, functional, or deontic. MOISE+ [63] is an extension of MOISE that clearly distinguishes these three dimensions along which an organization is structured.

The structural dimension specifies the roles, groups, and links in the organization. The functional dimension outlines how the global goals should be achieved by assigning different missions to different agents and creating a goal tree, often called a schema. Lastly, the deontic dimension integrates the functional and structural dimensions by specifying how each role in the structural dimension is obligated to pursue a set of goals provided by the functional dimension. An example of the specification of an organization whose goal is to write a paper is provided in Figure 6.2.



(c)

mission	cardinality
$mMan$	1..1
$mCol$	1..5
$mBib$	1..1

(d)

role	deontic relation	mission
editor	*permission*	$mMan$
writer	*obligation*	$mCol$
writer	*obligation*	$mBib$

Figure 6.2: Specification of an organization whose goal is to write a paper using MOISE+. Subfigure (a) represents the structural dimension, with the roles of editor and writer, both of which inherit from the author role, and a single group, wpgroup. Subfigures (b) and (c) represent the functional dimension. The overall goal of the organization is to write a paper, which is decomposed into a first draft version (fdv), further broken down into writing the title (wtitle), writing the abstract (wabs), and writing the section titles (wsectitles). Each of these leaf sub-goals is assigned to a different mission, such as mMan for the management of the organization, each with a specified cardinality in subfigure (c). Finally, subfigure (d) represents the deontic dimension, where editors are allowed to take on the mMan mission, while writers are obligated to take on their respective missions. Figure extracted from Hübner et al. [64].

Agents and Artifacts Framework

Ricci et al. [65] introduced the concept of artifacts to the modeling of organizations using MAS. In human societies and organizations, individuals are surrounded and empowered by various tools and artifacts that greatly enhance their problem-solving capabilities and facilitate all types of coordination and communication mechanisms. Bringing this idea to the fields of Distributed AI and MAS, Ricci et al. [65] introduced the Agents and Artifacts (A&A) Framework.

In this framework, the workspace of a given MAS organization is envisioned as a dynamic set of artifacts organized into different workspaces [65]. The central concept of the framework is that of an artifact, defined as anything created or used by agents to perform their tasks.

There are two main types of artifacts: resources and tools. A resource is an artifact used either as a primary source or as a target for some agent task. A tool, on the other hand, is an instrument an agent uses to execute a given task. Additionally, the concept of workspace serves to maintain the locality of artifacts by organizing the artifacts of the entire MAS into different environments.

Finally, artifact manuals are key for agents to be able to effectively use artifacts. These manuals contain information about how artifacts can be used and their intended purpose. Each manual should include a function description, detailing the intent of the artifact, a usage interface description, explaining how to interact with the artifact, and operating instructions, which provide guidance on how to use the artifact correctly.

ORA4MAS Framework.

This framework combines the concepts of Moise+ and the Agents and Artifacts Framework to provide a complete framework for modeling organizations. To achieve this, they specify four concrete artifacts that serve to coordinate the agents in the MAS organization. First is the OrgBoard, which is the main artifact of the organization and contains the rest of the artifacts. The OrgBoard contains a set of GroupBoards, each of which represents the notion of a group or department in a MAS organization. Each GroupBoard contains a set of playable roles to be adopted by the agents in that same GroupBoard, and a set of SchemaBoards. SchemaBoards are similar to missions, where each mission has a concrete goal that has been broken down into a hierarchical tree of subtasks. The SchemaBoard encapsulates all the information needed to accomplish the overarching task and each of the subtasks. Finally, the OrgBoard also contains a NormativeBoard, which establishes how each agent should behave, along with their constraints and restrictions.

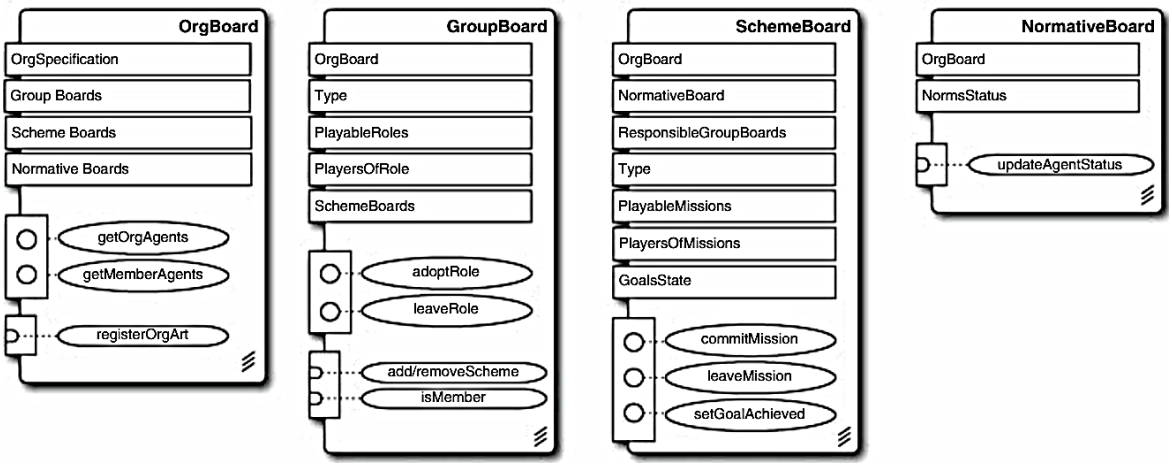


Figure 6.3: Artifacts used by the ORA4MAS Framework. Figure extracted from Hübner et al. [64].

We envision a future where the field of multi-agent LLMs leverages the concepts presented here to pave the way for more autonomous, scalable, and generally intelligent artificial problem-solving systems.

As LLM capabilities continue to advance, we foresee future frameworks in which LLMs can autonomously organize themselves across OrgBoards, GroupBoards, create their own SchemeBoards and Artifacts, and establish NormativeBoards to regulate the behavior of all agents within the LLM organization. We argue that the problem-solving potential of these LLM Organizations will grow at a much faster rate than that of individual LLMs, as the underlying *intelligence layer* continues to improve. Therefore, the potential of this approach will only expand as LLMs become increasingly intelligent.

Chapter 7

Conclusions

In this chapter, we summarize the key findings and contributions of this work, while reflecting on the broader implications and potential future directions in the development of multi-agent LLM frameworks.

7.1 Conclusions

The primary goal of this work was to explore the potential of multi-agent LLM frameworks in addressing some of the limitations of current LLMs, with the ultimate aim of advancing toward the creation of artificial general intelligence (AGI). Our main contribution has been the development of a meta-framework that can be instantiated with other multi-agent LLM frameworks to enhance their performance.

We began by providing a background on the key developments that have led to the rise of LLMs, including the advent of deep learning, the introduction of transformer architectures, and the discovery of scaling laws. Additionally, we offered a brief overview of the field of Multi-Agent Systems (MAS), highlighting its relevance to the LLM landscape.

In the literature review, we examined the recent emergence of LLM agents and multi-agent LLM frameworks. We analyzed two prominent examples of these frameworks and proposed a categorization of current multi-agent LLM systems based on five key dimensions: profiling, planning, memory, environment, actions, and communication.

Next, we provided an overview of the MACM and AgentCoder multi-agent LLM frameworks, along with LangChain and LangGraph, two coding frameworks for building multi-agent LLM systems. This set the stage for our two primary contributions: AgentCoder+, an enhanced version of the AgentCoder framework, and the meta-framework, which we instantiated with both the MACM and AgentCoder+ frameworks.

In the experimental evaluation, we provided an overview of the two datasets used for benchmarking: MATH and HumanEval. We then evaluated the performance of the proposed frameworks against baseline models and observed clear improvements across both benchmarks.

Finally, in the concluding chapter, we outlined our vision for the future of multi-agent LLM frameworks. We argued that the field would benefit from a shift from an agent-centric approach to an organization-centric one, which we refer to as ”LLM Organizations.” This mirrors the evolution of Multi-Agent Organizations in MAS, where Organization Theory and MAS were synergistically combined to create more structured and scalable systems.

7.2 Limitations and Future Work

Several limitations and areas for future work include:

- **Limited range of LLMs tested:** Our experimental evaluation was conducted using GPT-4o and GPT-4o-mini as the underlying LLMs. Future research could explore the impact of our frameworks when applied to different LLMs, potentially revealing varying performance across models with different architectures or training data.
- **Benchmark variety and coverage:** Due to the high computational costs of testing on datasets like HumanEval and MATH, our evaluation was limited to these two benchmarks. Expanding future work to incorporate a broader range of benchmarks, and covering a higher percentage of problems within each dataset, could offer a more comprehensive assessment of our frameworks.
- **Effect of iteration count on performance:** Our meta-framework was evaluated using a fixed number of iterations. Further research could investigate how varying the number of iterations impacts the overall performance of the multi-agent LLM system, potentially identifying optimal iteration counts for different problem types.
- **Addressing more complex problems:** While the benchmarks we used present significant challenges for current AI systems, future work could evaluate how our frameworks perform on more complex tasks, such as advanced software engineering projects or real-world problem-solving scenarios.
- **Developing more scalable frameworks:** Although our meta-framework is designed to be general and applicable across a variety of problem-solving tasks, its scalability needs further investigation. We hypothesize that performance may plateau after a certain number of iterations, suggesting the need for the development of more scalable multi-agent LLM frameworks in future research.
- **Transitioning towards LLM Organizations:** Our discussion of transitioning towards LLM Organizations remains theoretical. Future work should focus on implementing these ideas, offering a potential pathway toward more scalable and efficient multi-agent LLM systems that operate within organizational structures and artifacts.

Bibliography

[1] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. T. Ribeiro, and Y. Zhang, "Sparks of artificial general intelligence: Early experiments with gpt-4," *arXiv preprint*, 2023.

[2] B. Lei, "Macm: Utilizing a multi-agent system for condition mining in solving complex mathematical problems," *arXiv preprint arXiv:2404.04735*, 2024.

[3] D. Huang, Q. Bu, J. M. Zhang, M. Luck, and H. Cui, "Agentcoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint arXiv:2312.13010*, 2023.

[4] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, "Measuring mathematical problem solving with the MATH dataset," *CoRR*, vol. abs/2103.03874, 2021.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, and et al, "Evaluating large language models trained on code," *arXiv preprint*, 2021.

[6] T. M. Mitchell, "Machine learning," 1997.

[7] R. J. Solomonoff, "A formal theory of inductive inference. part i," *Information and control*, vol. 7, no. 1, pp. 1–22, 1964.

[8] ——, "A formal theory of inductive inference. part ii," *Information and control*, vol. 7, no. 2, pp. 224–254, 1964.

[9] A. N. Kolmogorov, "Three approaches to the quantitative definition ofinformation'," *Problems of information transmission*, vol. 1, no. 1, pp. 1–7, 1965.

[10] J. Rissanen, "Modeling by shortest data description," *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.

[11] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of control, signals and systems*, vol. 2, no. 4, pp. 303–314, 1989.

[12] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural networks*, vol. 2, no. 5, pp. 359–366, 1989.

[13] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[15] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[16] A. Vaswani, "Attention is all you need," *arXiv preprint arXiv:1706.03762*, 2017.

[17] S. Hochreiter, "Long short-term memory," *Neural Computation MIT-Press*, 1997.

[18] D. Bahdanau, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.

[19] M.-T. Luong, "Effective approaches to attention-based neural machine translation," *arXiv preprint arXiv:1508.04025*, 2015.

[20] A. M. Turing, *Computing machinery and intelligence*. Springer, 2009.

[21] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[22] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[23] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[24] M. Reid, N. Savinov, D. Teplyashin, D. Lepikhin, T. Lillicrap, J.-b. Alayrac, R. Soricut, A. Lazaridou, O. Firat, J. Schrittwieser *et al.*, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint arXiv:2403.05530*, 2024.

[25] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[26] G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth *et al.*, "Gemini: a family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[27] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang *et al.*, "The rise and potential of large language model based agents: A survey," *arXiv preprint*, 2023.

[28] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton *et al.*, "Mastering the game of go without human knowledge," *nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[29] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel *et al.*, "Mastering atari, go, chess and shogi by planning with a learned model," *Nature*, vol. 588, no. 7839, pp. 604–609, 2020.

[30] S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. Novikov, G. Barth-maron, M. Giménez, Y. Sulsky, J. Kay, J. T. Springenberg, T. Eccles, J. Bruce, A. Razavi, A. Edwards, N. Heess, Y. Chen, R. Hadsell, O. Vinyals, M. Bordbar, and N. de Freitas, "A generalist agent," *Transactions on Machine Learning Research*, 2022.

[31] S. Gravitas, "Auto-GPT," https://github.com/Significant-Gravitas/Auto-GPT, 2023.

[32] Y. Nakajima, "BabyAGI," https://github.com/yoheinakajima/babyagi, 2023.

[33] M. Minsky, *The Society of Mind*. Simon and Schuster, 1988.

[34] T. Händler, "Balancing autonomy and alignment: A multi-dimensional taxonomy for autonomous llm-powered multi-agent architectures," *arXiv preprint arXiv:2310.03659*, 2023.

[35] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan *et al.*, "Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors," *arXiv preprint*, 2023.

[36] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang *et al.*, "Metagpt: Meta programming for multi-agent collaborative framework," *arXiv preprint*, 2023.

[37] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[38] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[39] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, pp. 1–26, 2024.

[40] P. Zhao, Z. Jin, and N. Cheng, "An in-depth survey of large language model-based artificial intelligence agents," *arXiv preprint arXiv:2309.14365*, 2023.

[41] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[42] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," *arXiv preprint arXiv:2402.01680*, 2024.

[43] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[44] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk *et al.*, "Graph of thoughts: Solving elaborate problems with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 682–17 690.

[45] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, "Llm+ p: Empowering large language models with optimal planning proficiency," *arXiv preprint arXiv:2304.11477*, 2023.

[46] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[47] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[48] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar *et al.*, "Inner monologue: Embodied reasoning through planning with language models," *arXiv preprint arXiv:2207.05608*, 2022.

[49] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[50] N. Shinn, B. Labash, and A. Gopinath, "Reflexion: an autonomous agent with dynamic memory and self-reflection," *arXiv preprint arXiv:2303.11366*, 2023.

[51] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun, "Communicative agents for software development," *arXiv preprint arXiv:2307.07924*, 2023.

[52] H. Zhang, W. Du, J. Shan, Q. Zhou, Y. Du, J. B. Tenenbaum, T. Shu, and C. Gan, "Building cooperative embodied agents modularly with large language models," *arXiv preprint arXiv:2307.02485*, 2023.

[53] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[54] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[55] Z. Yang, L. Li, J. Wang, K. Lin, E. Azarnasab, F. Ahmed, Z. Liu, C. Liu, M. Zeng, and L. Wang, "Mm-react: Prompting chatgpt for multimodal reasoning and action," *arXiv preprint arXiv:2303.11381*, 2023.

[56] D. Surís, S. Menon, and C. Vondrick, "Vipergpt: Visual inference via python execution for reasoning," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 11 888–11 898.

[57] C. Zhang, K. Yang, S. Hu, Z. Wang, G. Li, Y. Sun, C. Zhang, Z. Zhang, A. Liu, S.-C. Zhu *et al.*, "Proagent: Building proactive cooperative ai with large language models," *arXiv e-prints*, pp. arXiv–2308, 2023.

[58] Y. Chang, K. Lo, T. Goyal, and M. Iyyer, "Booookscore: A systematic exploration of book-length summarization in the era of llms," *arXiv preprint arXiv:2310.00785*, 2023.

[59] J. Ferber, O. Gutknecht, and F. Michel, "From agents to organizations: an organizational view of multi-agent systems," in *International workshop on agent-oriented software engineering*. Springer, 2003, pp. 214–230.

[60] H. Abbas, S. Shaheen, and M. Amin, "Organization of multi-agent systems: An overview," *International Journal of Intelligent Information Systems*, 06 2015.

[61] J. Ferber, O. Gutknecht, F. Michel *et al.*, "Agent/group/roles: Simulating with organizations," in *Fourth International Workshop on Agent-Based Simulation (ABS03)*, 2003.

[62] M. Hannoun, O. Boissier, J. S. Sichman, and C. Sayettat, "Moise: An organizational model for multi-agent systems," in *Ibero-American Conference on Artificial Intelligence*. Springer, 2000, pp. 156–165.

[63] J. F. Hübner, J. S. Sichman, and O. Boissier, "Moise+ towards a structural, functional, and deontic model for mas organization," in *Proceedings of the first international joint conference on Autonomous agents and multiagent systems: part 1*, 2002, pp. 501–502.

[64] J. F. Hübner, O. Boissier, R. Kitio, and A. Ricci, "Instrumenting multi-agent organisations with organisational artifacts and agents: "giving the organisational power back to the agents"," *Autonomous agents and multi-agent systems*, vol. 20, no. 3, pp. 369–400, 2010.

[65] A. Ricci, M. Viroli, and A. Omicini, "Carta go: A framework for prototyping artifact-based environments in mas," in *International Workshop on Environments for Multi-Agent Systems*. Springer, 2006, pp. 67–86.

Appendix A

Prompts

A.1 AgentCoder+ Prompts

A.1.1 Agent: Test Generator

Listing A.1: Test Generator Prompts

```
SYSTEM PROMPT:
As a tester, your task is to create comprehensive test cases for the incomplete
     function. Here is an example
## Prompt 1:
‘‘‘python
from typing import List
def has_close_elements(numbers: List[float], threshold: float) -> bool:
    """ Check if in given list of numbers, are any two numbers closer to each
        other than
    given threshold.
    >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
    False
    >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
‘‘‘
## Completion 1:
‘‘‘python
assert has_close_elements([1.0, 2.0, 3.0], 0.5) == False
assert has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)== True
‘‘‘

USER PROMPT:
## Prompt:\n{method_description}\n ## Completion:\n
```

A.1.2 Agent: Test Judge

Listing A.2: Test Judge Prompts

```
SYSTEM PROMPT:
Your task is to evaluate the correctness of the tests provided and provide
    feedback on how to fix the tests if any of them are incorrect.
Report the total number of tests executed and the number of tests that passed.

USER PROMPT:
```

```
## Description of the method to test:\n{method_description}\n
## Tests to check:\n{generated_tests}\n
## Feedback in structured format:\n
```

A.1.3 Agent: Test Refiner

Listing A.3: Test Refiner Prompts

```
SYSTEM PROMPT:
You are an expert tester. Another tester has written some tests that are not
    correct. Your task is to refine the tests so that they are correct.

USER PROMPT:
## Description of the method to test:\n{method_description}\n
## Tests to Refine:\n{generated_tests}\n
## Feedback on the tests after running the tests:\n{feedback}\n
## Improved Tests:\n
```

A.2 MetaMACM Prompts

A.2.1 Agent: Test Generator

Listing A.4: Test Generator Prompts

```
SYSTEM PROMPT:
You are going to be given a potential solution to a math problem, called
    solution. Your task is to generate the minimal set of verifications the
    solution to the given math problem should pass. Focus only on verifying the
     core components of the problem, such as key equations, function
    evaluations, or any other specific operations described in the problem
    statement. Ensure that the tests confirm the correctness of these core
    components without introducing additional edge cases or variations. There
    should be no more than 2 or 3 verifications at most. Here are two examples:
## Example 1
### Math Problem:
If \(7 - 4x = 15\), what is the value of \(8x + 2\)?
### Tests a potential solution should pass:
1. Compute x = (solution - 2) / 8. Check that 7 - 4x = 15.
## Example 2
### Math Problem:
Every student in the senior class is taking history or science. There are $200$
     seniors in the class. If there are $126$ seniors taking history and $129$
    seniors taking science, how many students are taking both history and
    science?
### Tests a potential solution should pass:
1. Check that 126 + 129 - solution = 200.

USER PROMPT:
### Math Problem :\n{math_problem}\n
### Tests a potential solution should pass:\n
```

A.2.2 Agent: Test Judge

Listing A.5: Test Judge Prompts

```
SYSTEM PROMPT:
Your task is to evaluate the correctness of the tests provided and provide
    feedback on how to fix the tests if any of them are incorrect. Report the
    total number of tests executed and the number of tests that are correct in
    your opinion.

USER PROMPT:
## Description of the math problem whose solution we will want to test:\n{
    math_problem}\n
## Tests to check: \n{generated_tests}\n
## Feedback about the correctness of the tests in structured format:\n
```

A.2.3 Agent: Test Refiner

Listing A.6: Test Refiner Prompts

```
SYSTEM PROMPT:
You are an expert tester. Another tester has written some tests to check if a
    solution to a given math problem is correct, but the tests are not correct.
     Your task is to refine the tests so that they are correct and a correct
    solution to the problem would pass the tests. Do not attempt to solve the
    problem, just refine the tests.

USER PROMPT:
## Description of the math problem for whose solution we are designing tests:\n
    {math_problem}
## Tests to Refine:\n{generated_tests}
## Feedback on the tests after running the tests:\n{feedback}\n
## Improved Tests:\n
```

A.2.4 Agent: Solution Evaluator

Listing A.7: Solution Evaluator Prompts

```
SYSTEM PROMPT:
Your task is to evaluate the final solutions provided by the agents and select
    the best solution. You should report the number of tests that a given
    solution passes.

USER PROMPT:
## Candidate for being the solution to the problem:\n{answer}\n
{tests}\n
---
## Report of the number of tests that the solution passes and the total number
    of tests:\n
```

A.3 MetaAgentCoder+ Prompts

A.3.1 Agent: Solution Evaluator

Listing A.8: Solution Evaluator Prompts

```
SYSTEM PROMPT:
Your task is to evaluate the final solutions provided by the agents and select
    the best solution. You should report the number of tests that a given
    solution passes.
```

```
USER PROMPT:
## Code to test (Do not modify): \n{completed_method}\n {test_cases}\n\n ---\n
    ## Report of the number of tests that the solution passes and the total
    number of tests: \n
```